



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

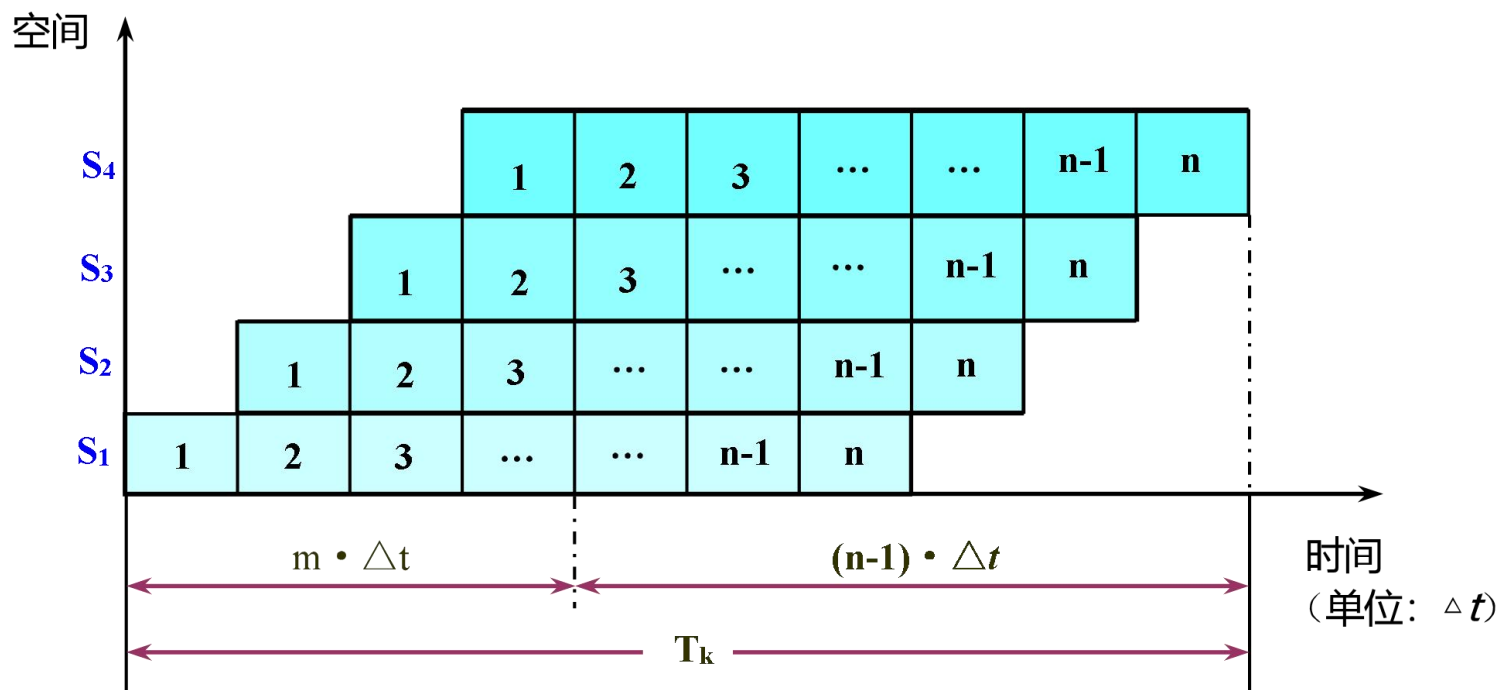
第四章 向量处理机

2026年春季学期

标量处理机问题

第 2 页

指令的流水执行机制：

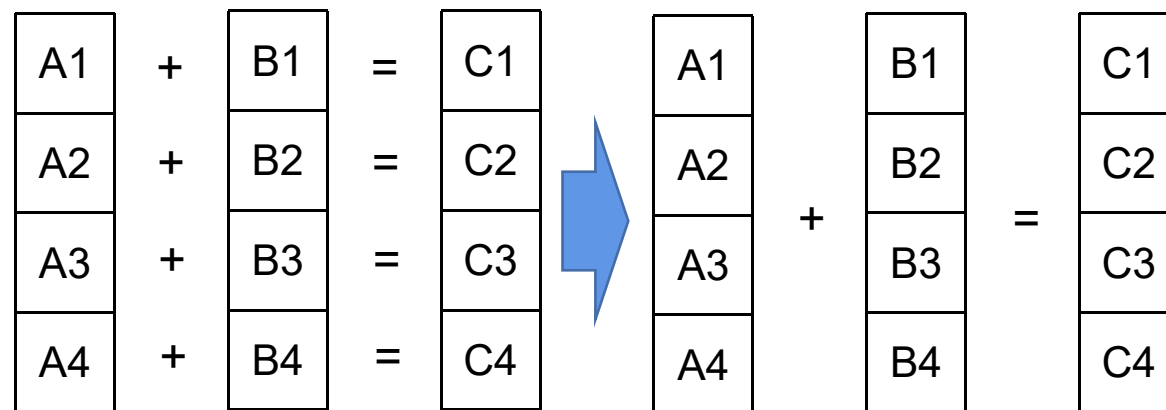
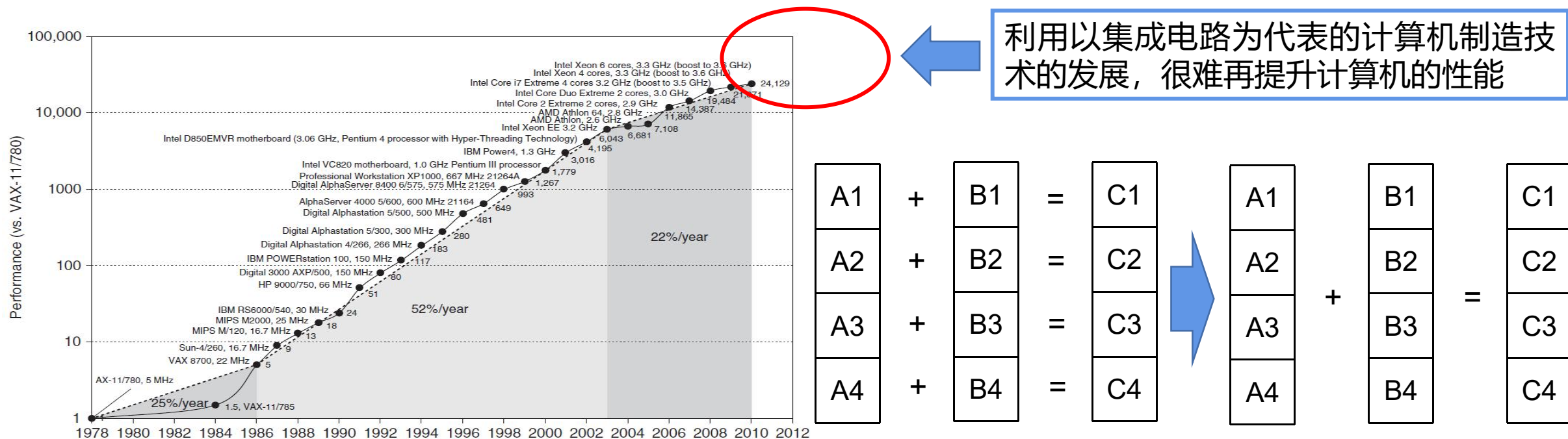


流水线调度和流水线冲突解决提升了流水线的效率，从而减少了程序的执行时间
存在问题：一个周期只能执行一次标量数据运算，执行效率低

标量处理机问题

第 3 页

- 不具有向量数据表示和相应的向量指令的流水线处理机，称为标量处理机。



SISD (单指令单数据)

SIMD (单指令多数据)

效率提升

向量计算需求

第 4 页

- 向量是一个既有大小、又有方向的量

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix} \begin{bmatrix} b_{1,1} & b_{1,2} & \cdots & b_{1,j} \\ b_{2,1} & b_{2,2} & \cdots & b_{2,j} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n,1} & b_{n,2} & \cdots & b_{n,j} \end{bmatrix} = \begin{bmatrix} \sum_{k=1}^n a_{1,k} b_{k,1} & \sum_{k=1}^n a_{1,k} b_{k,2} & \cdots & \sum_{k=1}^n a_{1,k} b_{k,j} \\ \sum_{k=1}^n a_{2,k} b_{k,1} & \sum_{k=1}^n a_{2,k} b_{k,2} & \cdots & \sum_{k=1}^n a_{2,k} b_{k,j} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^n a_{m,k} b_{k,1} & \sum_{k=1}^n a_{m,k} b_{k,2} & \cdots & \sum_{k=1}^n a_{m,k} b_{k,j} \end{bmatrix}$$

- 向量由一组有序、具有相同类型和位数的元素组成。
- 专业应用：科学计算、图像处理、人工智能

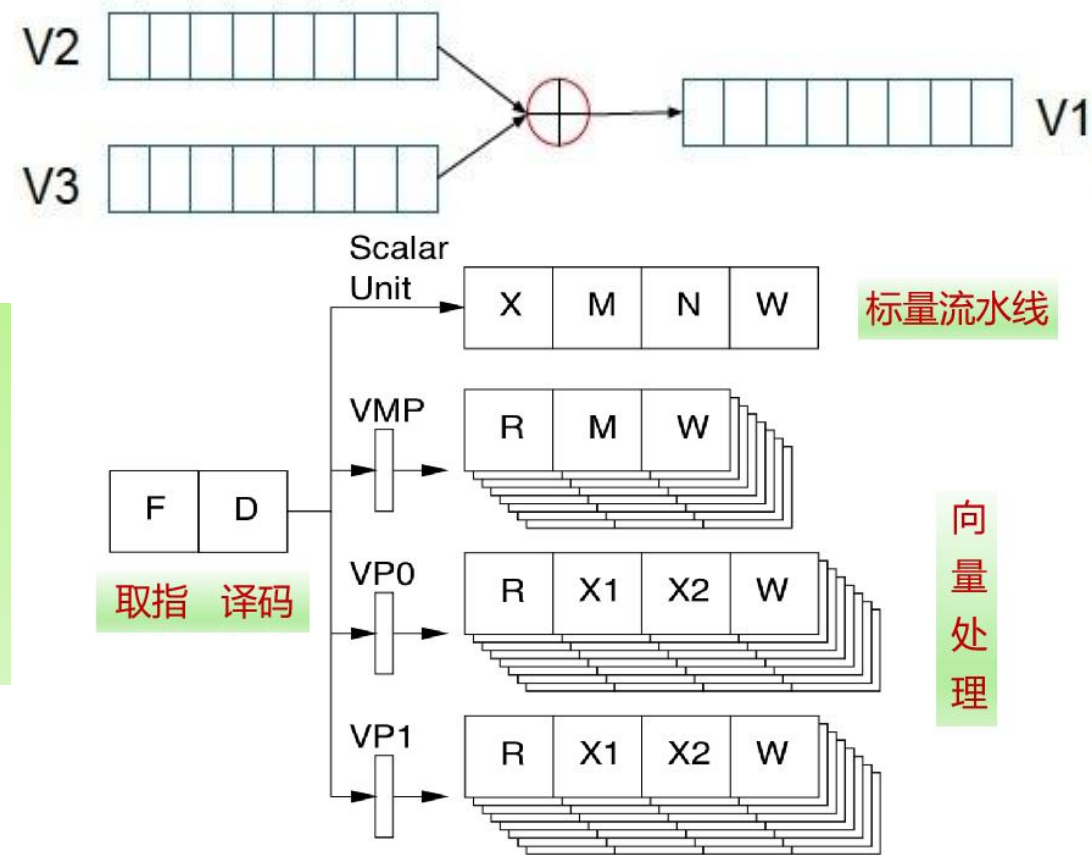
标量处理机与向量处理机

第 5 页

- 标量处理机：指只有标量数据表示和标量指令系统的处理机
- 向量处理机：同时具有标量数据表示和向量数据表示，以及标量指令系统与向量指令系统的处理机。

例如，向量指令 `ADDV V1, V2, V3`

指令	操作数	操作	说明
<code>ADDV</code>	<code>V1, V2, V3</code>	<code>V1 = V2 + V3</code>	vector + vector
<code>ADD_SV</code>	<code>V1, <u>F0</u>, V2</code>	<code>V1 = <u>F0</u> + V2</code>	scalar + vector
<code>MULTV</code>	<code>V1, V2, V3</code>	<code>V1 = V2 x V3</code>	vector x vector
<code>MULSV</code>	<code>V1, <u>F0</u>, V2</code>	<code>V1 = <u>F0</u> x V2</code>	scalar x vector

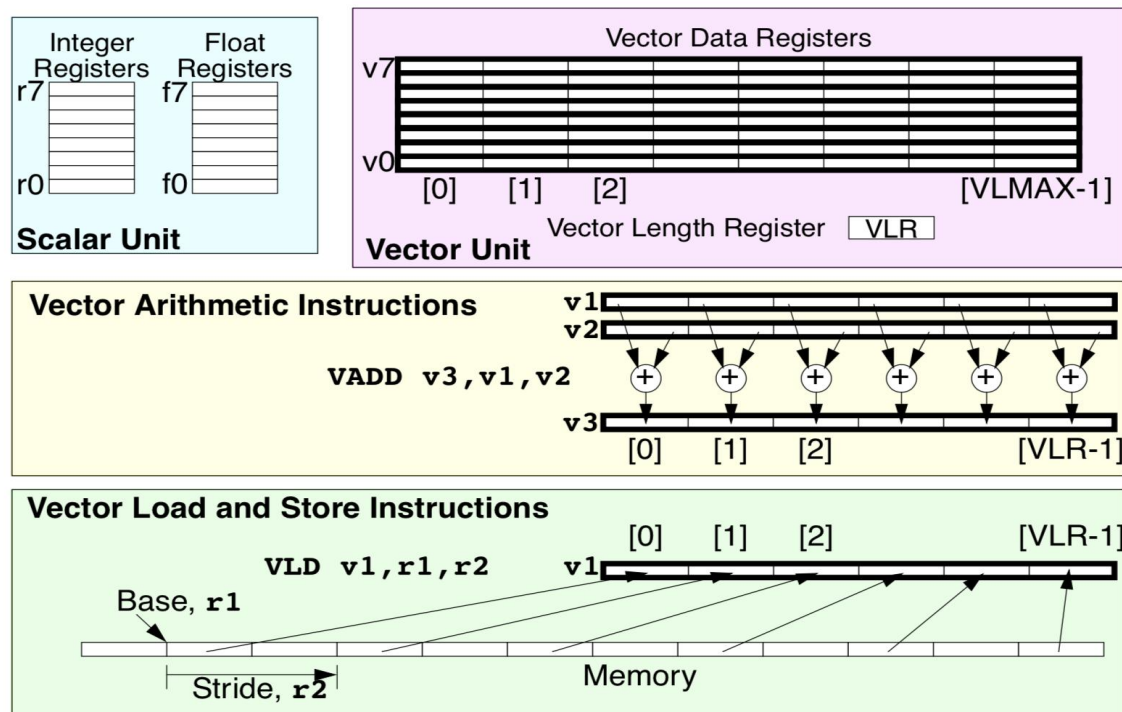


标量处理机与向量处理机

第 6 页

■ 典型的向量处理机

- 1976年 Cray-1超级计算机，浮点运算速度达到了每秒1亿次
- CDC Cyber 205, Cray Y-MP, NEC SX-X/44, Fujitsu VP2600等，性能达到了每秒几十亿~几百亿次浮点运算



- 4. 1 [向量的处理方式](#)
- 4. 2 [向量处理机的结构](#)
- 4. 3 [提高向量处理机性能的常用技术](#)
- 4. 4 [向量处理机的性能评价](#)
- 4. 5 [向量处理机实例](#)

4. 1 向量的处理方式

4.1.1 向量数据的表示方式

第 9 页

C 代码

```
for (i=0; i<64; i++)  
    C[i] = A[i] + B[i];
```

标量代码

```
    li x4, 64  
loop:  
    fld f1, 0(x1)  
    fld f2, 0(x2)  
    fadd.d f3,f1,f2  
    fsd f3, 0(x3)  
    addi x1, 8  
    addi x2, 8  
    addi x3, 8  
    subi x4, 1  
    bnez x4, loop
```

向量代码

```
li x4, 64  
setvl x4  
vld    v1, x1  
vld    v2, x2  
vadd   v3,v1,v2  
vst    v3, x3
```

- 在标量处理机上用10条指令，其中有9条指令要循环64次；采用多寄存器结构的两地址指令编写程序，存储器采用字节编址方式，字长为32位。
- 一条向量指令可以处理N个或N对操作数，性能高

4.1.1 向量计算需求

第 10 页

向量指令集的优点

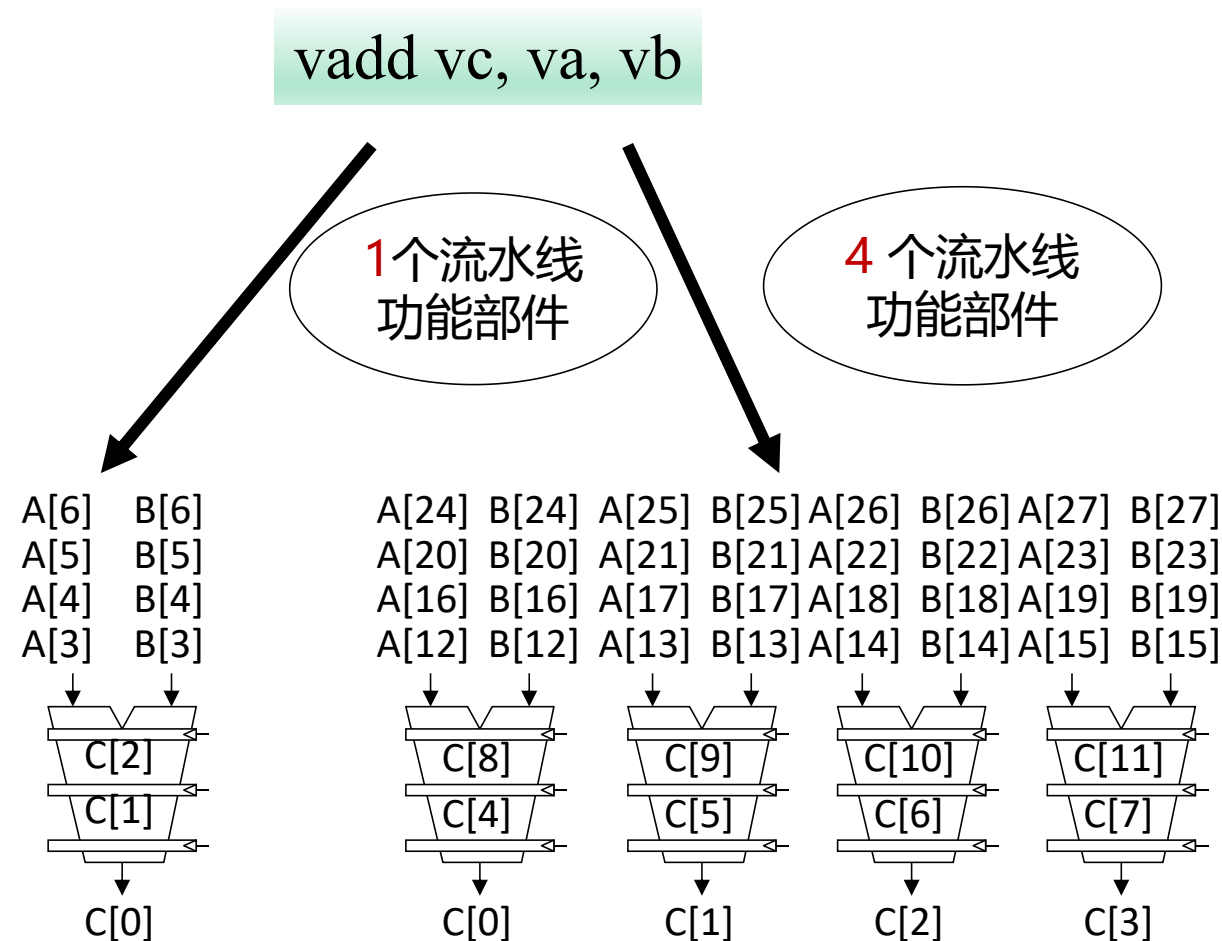
◆ 精简

- 一个短的指令，包含N个操作

◆ 表示能力强 告诉硬件 N 个操作：

- 不相关
- 使用相同功能部件 (FU)
- 使用独立的 (disjoint) 寄存器
- 访问连续的存储块 (单步 L/S)
- 访存模式确定 (跨步 L/S)

◆ 可扩展



4.1.1 向量数据的表示方式

第 11 页

- 向量处理机是解决数值计算问题的一种高性能计算机结构
- 实际的应用问题中通常既有向量计算又有标量计算，而且两类计算有一定的比例
 - 实现方式一：向量处理机通常属大型或巨型机，一般向量计算机中包括有一台高性能标量处理机
 - 实现方式二：可以用微机加一台向量协处理器组成

如何平衡向量硬件设备和标量硬件设备的利用率以达到最优？

4.1.1 向量数据的表示方式

第 12 页

向量与标量性能的平衡：

■向量平衡点(vector balance point)：为了使向量硬件设备和标量硬件设备的利用率相等，一个程序中向量代码所占的百分比。

■关键问题是：希望向量硬件和标量硬件都能够充分利用。

例如：一个系统的向量运算速度为90Mfolps，标量运算速度为10Mfolps。如果程序的90%是向量运算，10%是标量运算。则向量平衡点为0.9。硬件利用率最高。

■向量平衡点必须与用户程序的向量化程度相匹配。

IBM向量计算机的设计思想与上述方法不同，它维持较低的向量与标量比例，定在3 ~ 5的范围之间。

这种做法能够适应通用应用问题对标量和向量处理要求

4.1.1 向量数据的表示方式

第 13 页

- 如何发挥向量处理机的计算优势，关键是必须把问题转化为向量运算？

关键点一：从标量表示到向量数据的表示方式：

1. 等间距向量表示法
2. 带位移量的向量表示法
3. 稀疏向量表示法

4.1.1 向量数据的表示方式

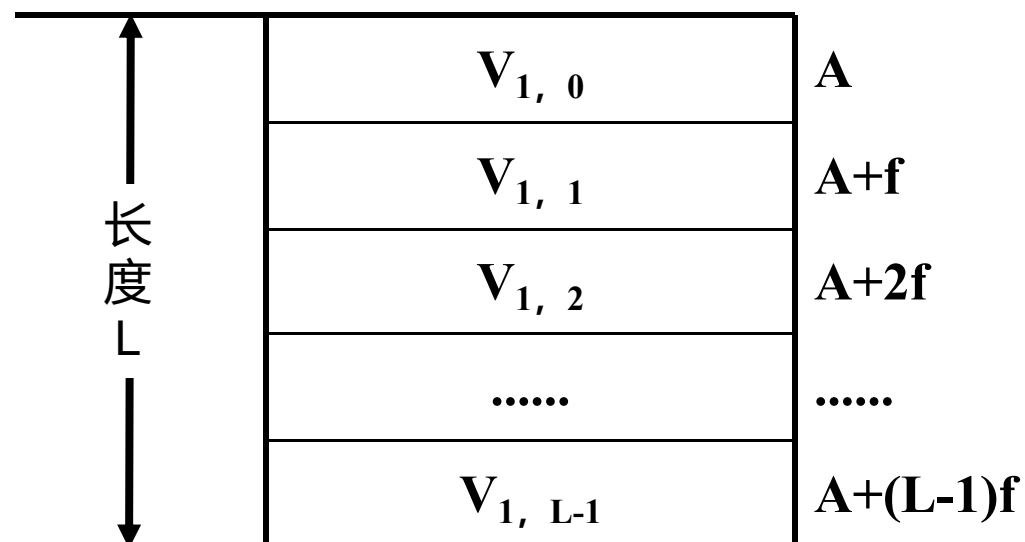
第 14 页

1. 等间距向量表示法

① 向量起始地址 A

② 向量长度 L

③ 向量间距 f



4.1.1 向量数据的表示方式

第 15 页

1. 等间距向量表示法

例如：我国研制的银河向量机，有8个向量寄存器：V0 ~ V7每个向量寄存器由64个64位的寄存器组成。存储器字长64位，采用字节编址方式，则连续向量的间距为 $f=8$ 。向量指令采用三地址形式。

向量指令格式	OP	i	j	k
向量加法指令	VADD	1	3	5

例如： $V_i \leftarrow V_j \text{ OP } V_k$ ，向量长度 (VL) = 50，则实际完成的运算是：

V3,00 ~ V3,49与V5,00 ~ V5,49分别相加，结果放在V1,00 ~ V1,49中。

4.1.1 向量数据的表示方式

第 16 页

2. 带位移量的向量表示法

- ① 向量起始地址 A
- ② 向量长度 L
- ③ 向量位移量 f
- ④ 向量有效长度 $L-f$
- ⑤ 向量起始地址 $A+f$



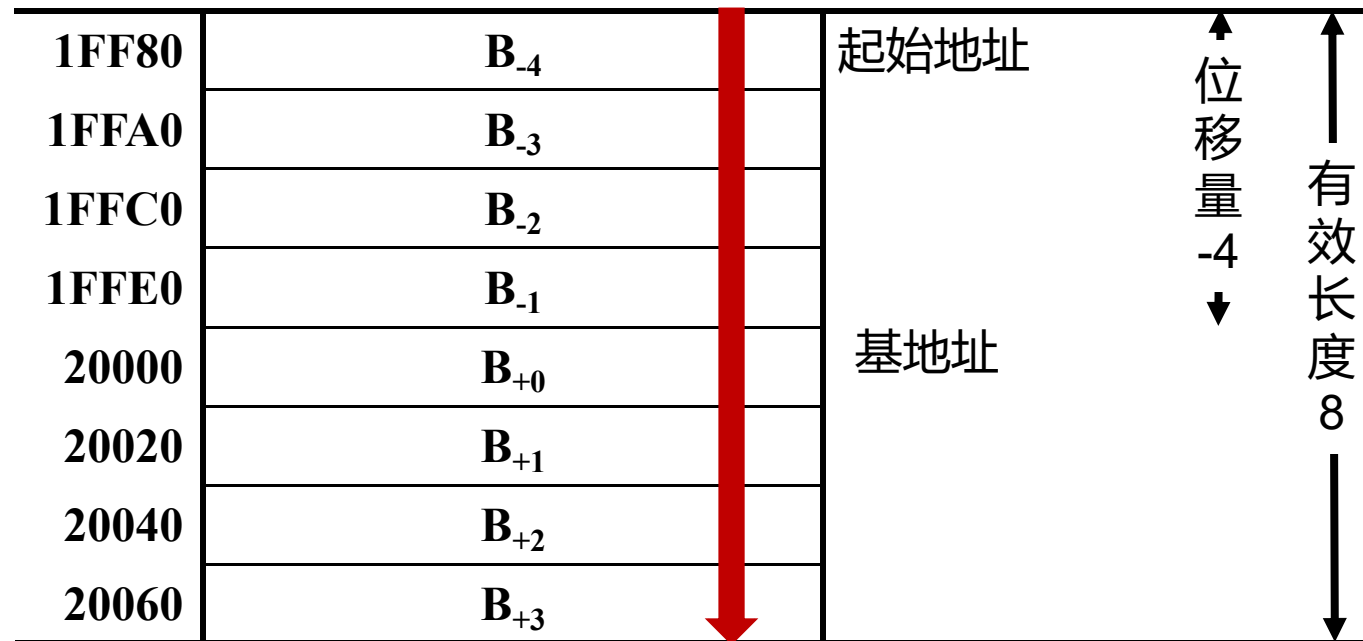
A向量: $A=10000$, $L=12$, $f=4$

4.1.1 向量数据的表示方式

第 17 页

2. 带位移量的向量表示法

- ① 向量起始地址 A
- ② 向量长度 L
- ③ 向量位移量 f
- ④ 向量有效长度 $L-f$
- ⑤ 向量起始地址 $A+f$



B向量: $B=20000$, $L=4$, $f=-4$

优点: 每个向量可以带有位移, 通过控制向量实现可变增量; 能够表示稀疏向量。

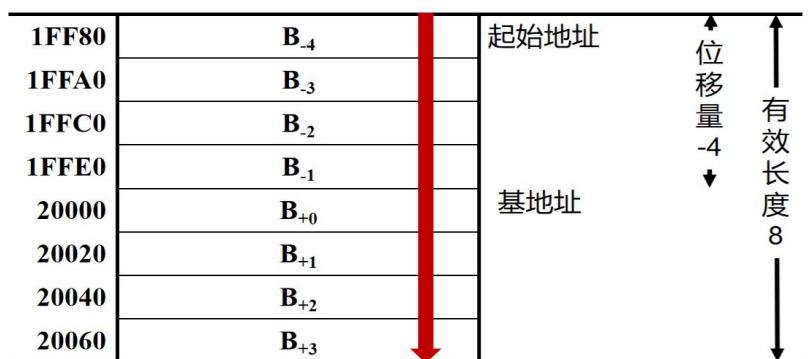
4.1.1 向量数据的表示方式

第 18 页

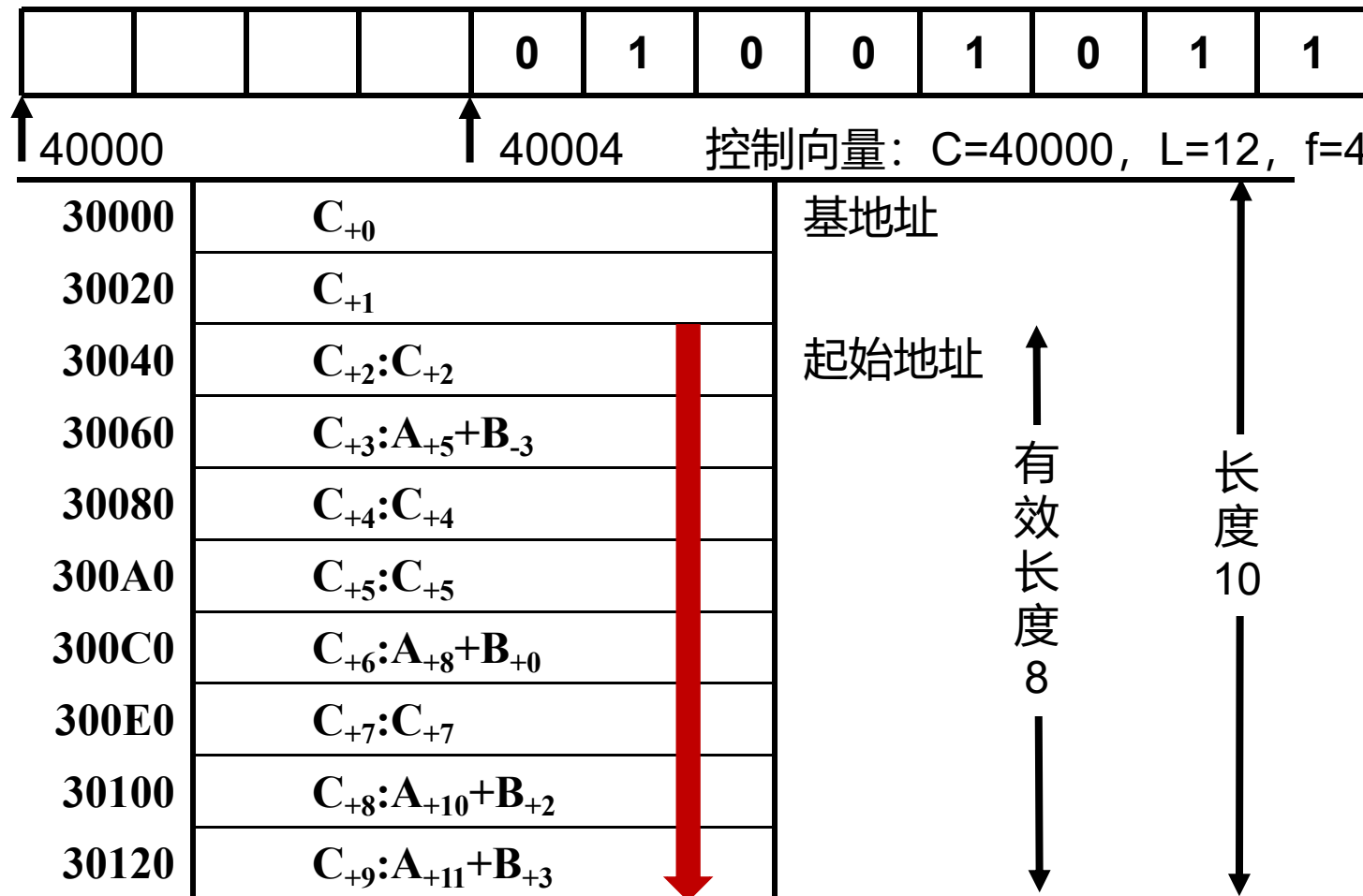
2. 带位移量的向量表示法 (计算 $C=A+B$)



A向量: $A=10000$, $L=12$, $f=4$



B向量: $B=20000$, $L=4$, $f=-4$



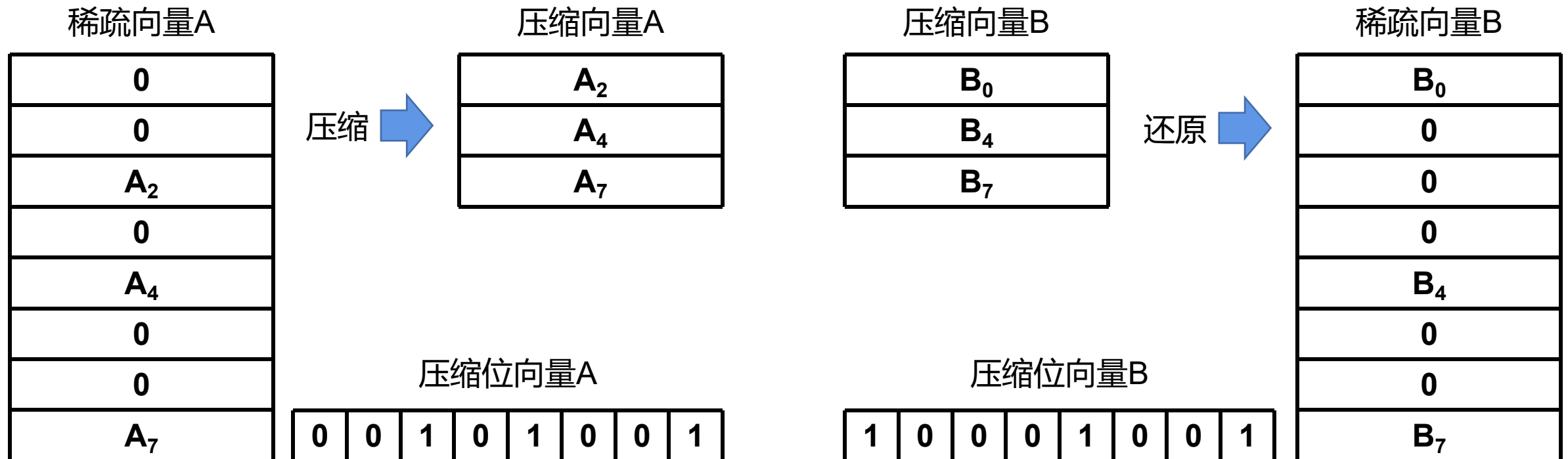
C向量: $C=30000$, $L=10$, $f=2$

4.1.1 向量数据的表示方式

第 19 页

3. 稀疏向量表示法

- 定义：稀疏向量，0元素很多
- 采用压缩存储稀疏向量可以节省存储空间

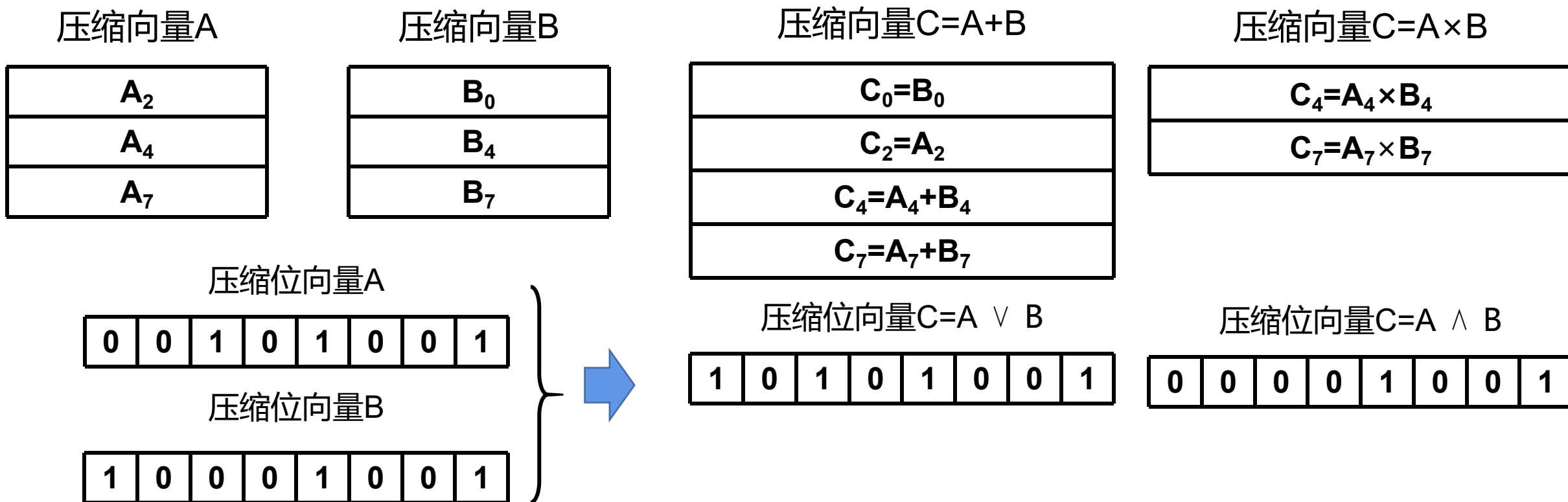


4.1.1 向量数据的表示方式

第 20 页

3. 稀疏向量表示法

■ 计算方法 ① 可以还原之后进行运算；② 也可以用压缩方法直接进行运算



4.1.2 向量的处理方式

第 21 页

一个简单的C语言程序

```
for ( i = 1 ; i <= 100 ; i++ )  
    D[i] = A[i] * (B[i] + C[i])
```

关键点二：在向量处理机上实现有**3**种处理方式：

- 1. 横向处理方式**，又称水平处理方式，横向加工方式等，向量计算是按行的方式从左至右横向地进行
- 2. 纵向处理方式**，又称垂直处理方式，纵向加工方式等，向量计算是按列的方式自上而下纵向地进行
- 3. 纵横处理方式**，又称分组处理方式，纵横加工方式等，横向处理和纵向处理相结合的方式

4.1.2 向量的处理方式

第 22 页

1. 横向处理方式，又称水平处理方式，横向加工方式等，按行的方式逐个分量进行处理。假设中间结果为 $T[i]$

计算第1个分量: $T[1]=B[1]+C[1]$

$D[1]=A[1]\times T[1]$

计算第2个分量: $T[2]=B[2]+C[2]$

$D[2]=A[2]\times T[2]$

.....

计算最后一个分量: $T[100]=B[100]+C[100]$

$D[100]=A[100]\times T[100]$

存在两个问题

- ① 在计算向量的每个分量时，都发生写数据相关，流水线效率低；如果采用多功能流水线，必须频繁进行流水线切换
- ② 横向处理方式对向量处理机不适合。即使在标量处理机中，也经常通过编译进行指令流高度

4.1.2 向量的处理方式

第 23 页

2. **纵向处理方式**，又称垂直处理方式，纵向加工方式等，按列的方式逐个分量进行处理。假设中间结果为 $T(i)$


$$T[1] = B[1] + C[1]$$

$$T[2] = B[2] + C[2]$$

.....

$$T[100] = B[100] + C[100]$$

$$D[1] = A[1] \times T[1] \quad // \text{中间结果}$$

$$D[2] = A[2] \times T[2]$$

.....

$$D[100] = A[100] \times T[100]$$

采用向量指令只需2条：

VADD B, C, T

VMUL A, T, D

- 这种处理方式适合向量处理机实现
- 数据相关不影响流水线连续工作
- 不同的运算操作只需要切换1次

4.1.2 向量的处理方式

第 24 页

3. 纵横向处理方式，又称分组处理方式，纵横加工方式等，分级按列的方式逐个分量进行处理。假设中间结果为 $T(i)$

现有问题：向量长度不固定，对存储系统要求高，且系统不灵活

解决方法：用于寄存器-寄存器结构的向量处理机中，向量寄存器的长度是有限的

当向量长度 N （例100）大于向量寄存器长度 n （例如8、16等）时，需要分组处理。

分组方式： $N = k \times n + r$ 其中： r 为余数，共分 $k+1$ 组

组内采用纵向处理方式，组间采用横向处理方式，因此也称为分组处理方式，纵横向加工方法等。

4.1.2 向量的处理方式

第 25 页

3. **纵横向处理方式**，又称分组处理方式，纵横加工方式等，分级按列的方式逐个分量进行处理。假设中间结果为 $T[i]$

运算过程为：

$$\text{第1组: } T[1,n] = B[1,n] + C[1,n] \quad D[1,n] = A[1,n] \times T[1,n]$$

$$\text{第2组: } T[n+1,2n] = B[n+1,2n] + C[n+1,2n] \quad D[n+1,2n] = A[n+1,2n] \times T[n+1,2n]$$

.....

$$\text{第}k+1\text{组: } T[kn+1,N] = B[kn+1,N] + C[kn+1,N] \quad D[kn+1,N] = A[kn+1,N] \times T[kn+1,N]$$

■ 每组用两条向量指令，每组发生相关1次，功能切换发生2次

4. 2 向量处理机的结构

4.2 向量处理机的结构

第 27 页

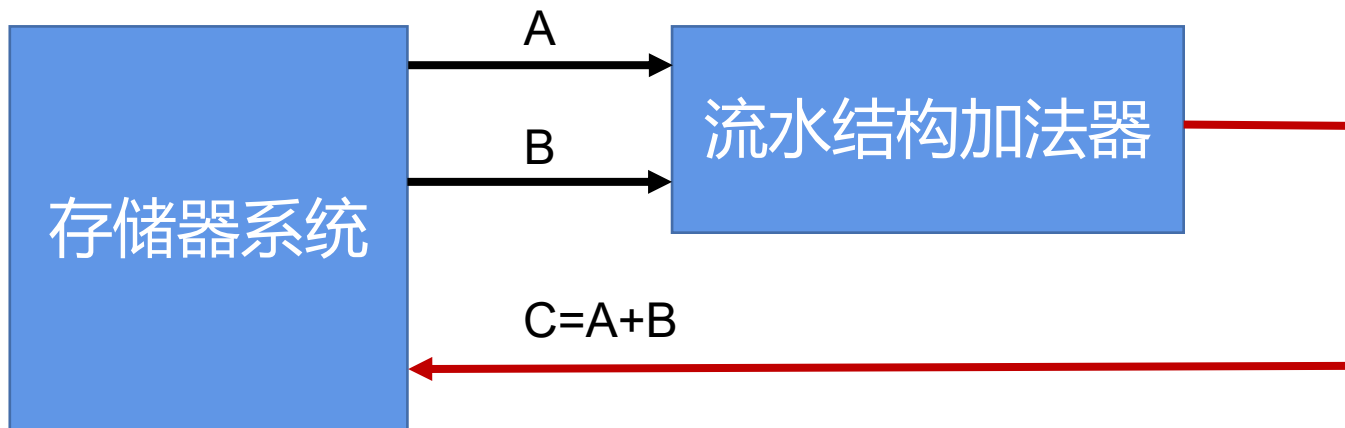
向量处理机的基本思想是把两个向量的对应分量进行运算，产生一个结果向量。

例：如果A、B、C都是向量，各有N个元素，则一台向量处理机能够完成如下运算：

$$C_i = A_i + B_i$$

$0 \leq i \leq N-1$ ，其中A、B、C用分量形式表示

采用流水线运算部件实现上述运算：



如果要求取操作数、运算、写结果回存储器在一个时钟周期内完成，则要求存储系统能在一个时钟周期内读出两个操作数和写回一个运行结果。

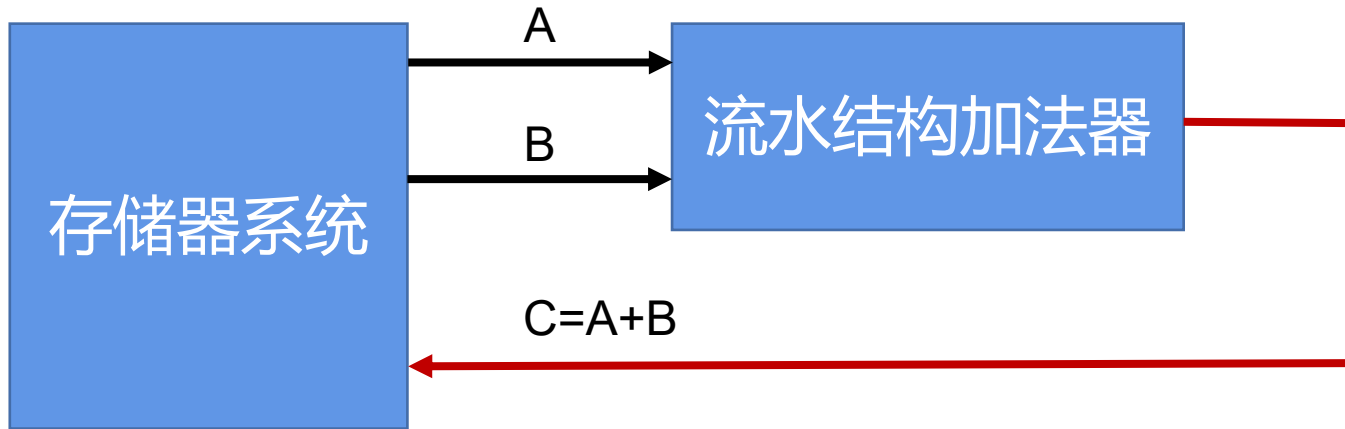


存储器系统的带宽至少应3倍于一般的存储器系统

4.2 向量处理机的结构

第 28 页

采用流水线运算部件实现上述运算：



存储系统结构和存储系统速度
是关键

向量处理机要解决的问题：

- 1) 维持连续的数据流（提供连续的A和B）
- 2) 降低对存储器带宽的压力

注：向量结构中取指令操作所要求的带宽可以忽略

4.2 向量处理机的结构

第 29 页

系统结构设计者所面临的主要问题是设计出能满足运算器带宽要求的存储器系统，主要采用两种方式：

方式一：利用几个独立的存储器模块来支持对相互独立的数据的并发访问，达到所要求的存储器带宽，即存储器—存储器结构。

方式二：构造一个具有所要求带宽的高速中间存储器，并能实现该高速中间存储器与主存储器之间的快速数据交换，即寄存器—寄存器结构。

4.2 向量处理机的结构

第 30 页

- 向量处理机的结构由所采用的向量处理方式决定，有两种典型结构
 - **存储器-存储器型结构**：纵向处理方式采用，利用几个独立的存储器模块来支持向对独立的数据并发访问
 - ✓ 多个独立得到存储模块并行工作
 - ✓ 处理机结构简单，对存储系统的访问速度要求很高
 - **寄存器-寄存器型结构**：分组处理方式采用，构造一个具有所要求带块的高速中间寄存器，实现高速中间寄存器与主寄存器之间的快速数据交换
 - ✓ 运算通过向量寄存器实现，需要大量高速寄存器
 - ✓ 对存储系统访问速度要求低

4.2.1 “存储器-存储器”结构

第 31 页

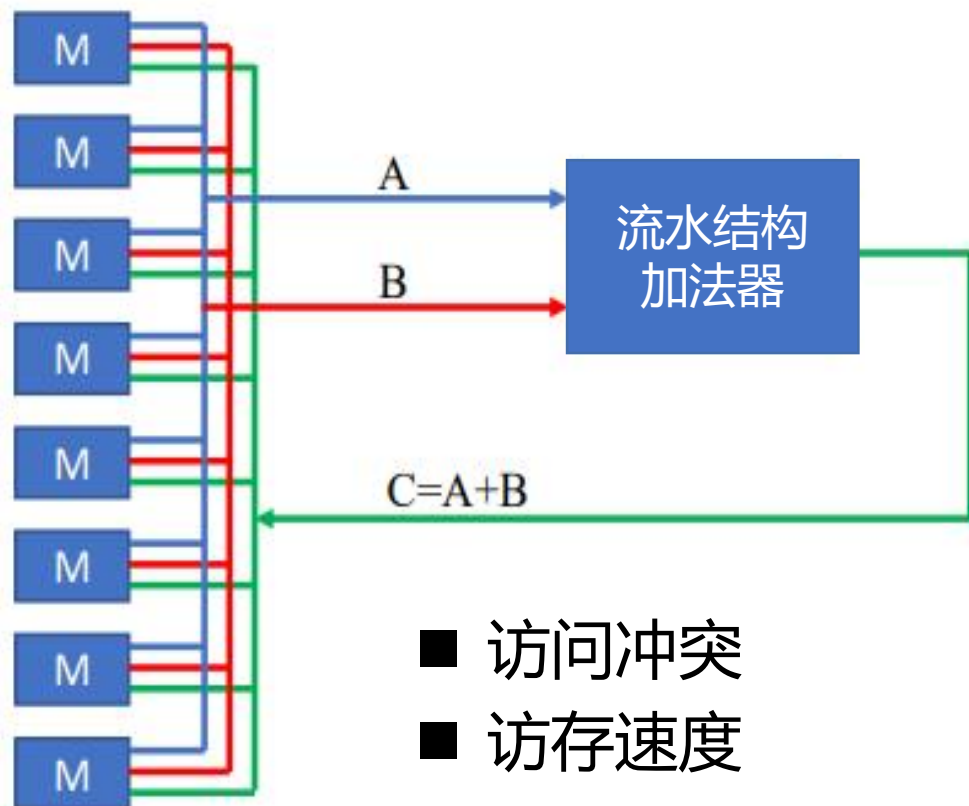
1. 采用纵向处理方式的向量处理机对处理机结构的要求：**存储器 - 存储器结构**

- 向量指令的源向量和目的向量都是存放在存储器中，运算的中间结果需要送回存储器。
- 流水线运算部件的输入和输出端都直接（或经过缓冲器）与存储器相联，从而构成存储器-存储器型操作的运算流水线。
 - ✓ 例如：STAR-100、CYBER-205

4.2 向量处理机的结构

第 32 页

“存储器 - 存储器”型操作的运算流水线设计要求：每拍从存储器读取两个数据，并向存储器写回一个结果。



存储器-存储器结构实例：

1. 一个具有由8个三端口存储器组成的存储器系统的向量处理机
2. 存储系统由8个模块构成，带宽是单个模块的8倍
3. 流水线运算器与主存储器之间有三条相互独立的数据通路，各个数据通路可以同时工作，但一个存储器模块在某一时刻只能为一个通路服务

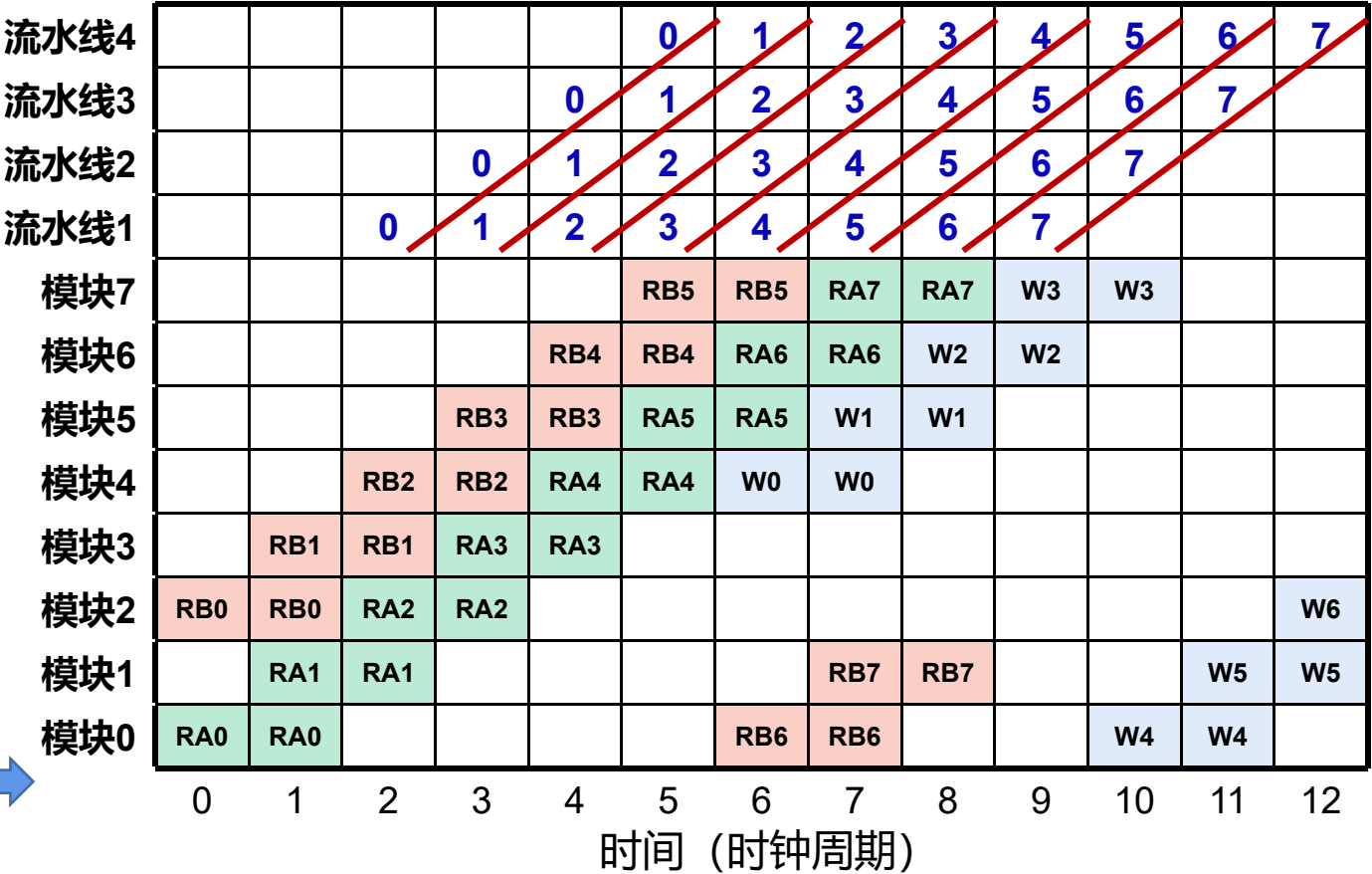
4.2 向量处理机的结构

例：假设一个由8个三端口存储器组成的存储器系统的，1个存储周期占2个处理器周期，则满足流水线所需的带宽至少是单个存储器模块的6倍，试分析该结构的具体执行时序图。

模块 0	A[0]		B[6]			C[4]	
模块 1	A[1]		B[7]			C[5]	
模块 2	A[2]	B[0]				C[6]	
模块 3	A[3]	B[1]				C[7]	
模块 4	A[4]	B[2]				C[0]	
模块 5	A[5]	B[3]				C[1]	
模块 6	A[6]	B[4]				C[2]	
模块 7	A[7]	B[5]				C[3]	

向量A、B、C在存储系统中存放的情况

设定两个向量在轮流方式下分量相加的时序图



4.2 向量处理机的结构

第 34 页

- 访问冲突问题：采用交叉访问方式，一个由n个存储体构成的主存储器，它的速度实际上并不能提高n倍，其根本原因是存在有**访问冲突**。
- 产生访问冲突的根源主要有两个：一是程序中有转移指令，二是**数据的随机性**。

体内地址 0号体 1号体 2号体 3号体

0	A0	A1	A2	A3
1	A4	A5	A6	A7
2	A8	A9	A10	A11
3	A12	A13	A14	A15

具体内容详见在存储系统设计。

一维数组的访问冲突分析：

1. 如果按连续地址访问，没有冲突。
2. 如果按位移量 $L=2$ 变址方式访问，频带宽
度降低一半，即可能有一半地址是冲突的。
 - 原因是传统的交叉访问存储器的存储体个数 n 为2的整数幂，因此变址位移量正好是 n 的约数。

地址 $A_i + m \times L$ 访问的存储器与地址 A_i 访问的存储器是一致的概率是50%

4.2 向量处理机的结构

第 35 页

解决访问冲突的方法之一：把存储体的个数 n 选为质数，且 $n \geq$ 向量长度，变址位移量就必然与 n 互质，一维数组的访问冲突自然也就不存在了。

- 例如美国研制的BSP巨型计算机的存储体个数为17，我国研制的银河巨型计算机的存储体个数为37。

$m \times L$ 不能整除 n



地址 $A_i + m \times L$ ($L=1 \sim 16$) 访问的存储器与地址 A_i 访问的存储器是一致的概率是0

体内地址	0号体	1号体	2号体	3号体	4号体
0	A0	A1	A2	A3	A4
1	A5	A6	A7	A8	A9
2	A10	A11	A12	A13	A14
3	A15	A16	A17	A18	A19

体内地址	0号体	1号体	2号体	3号体	4号体
0	A0	A1	A2	A3	A4
1	A5	A6	A7	A8	A9
2	A10	A11	A12	A13	A14
3	A15	A16	A17	A18	A19

$L=2$

$L=3$

4.2 向量处理机的结构

第 36 页

解决访问冲突的方法之一：把存储体的个数 n 选为质数，且 $n \geq$ 向量长度，变址位移量就必然与 n 互质，一维数组的访问冲突自然也就不存在了。

- 例如美国研制的BSP巨型计算机的存储体个数为17，我国研制的银河巨型计算机的存储体个数为37。

$m \times L$ 不能整除 n



地址 $A_i + m \times L$ ($L=1 \sim 16$) 访问的存储器与地址 A_i 访问的存储器是一致的概率是0

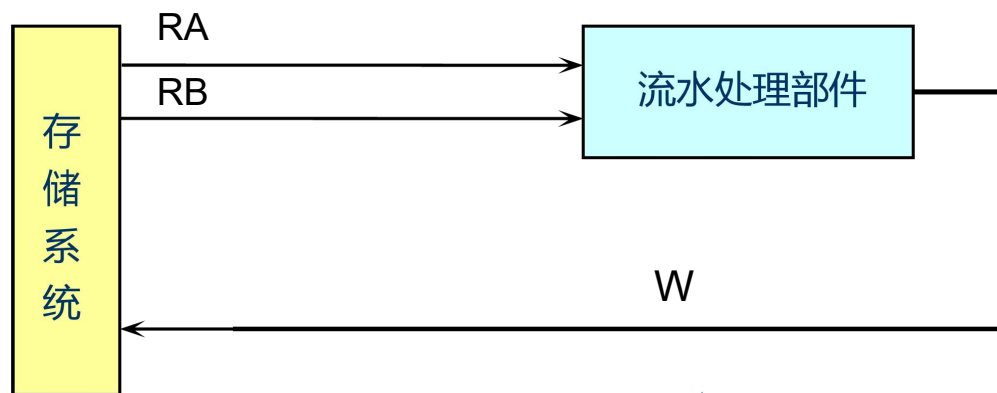
体内地址	0号体	1号体	2号体	3号体	4号体
0	A0	A1	A2	A3	A4
1	A5	A6	A7	A8	A9
2	A10	A11	A12	A13	A14
3	A15	A16	A17	A18	A19

存在问题：虽然采用质数个存储体能够缓解访问存储器的冲突，但是数据经过多次运算之后，在存储体中的分布必然发生改变，无法克服访问冲突问题。

4.2.1 “存储器-存储器”结构

第 37 页

解决访问冲突的方法之二：在运算流水线的输入端和输出端增加操作数**可变缓冲器**和写结果**可变缓冲器**，以便消除征用存储器的现象



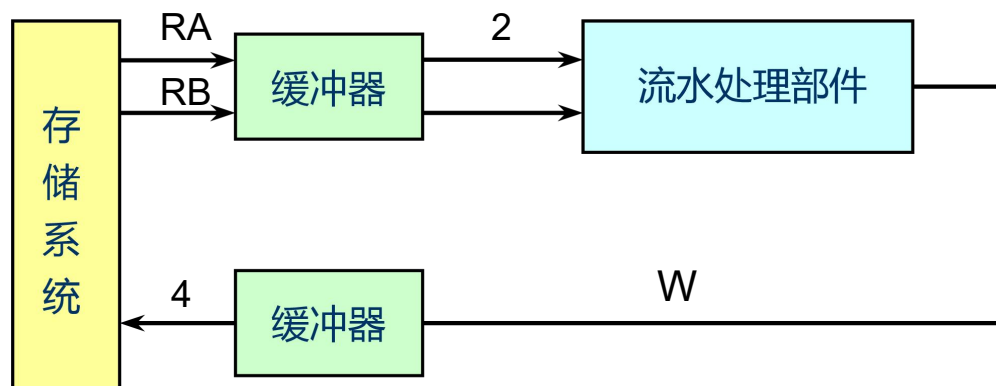
无法对齐

模块 0	A[0]	B[0]			C[0]		
模块 1	A[1]	B[1]			C[1]		
模块 2	A[2]	B[2]			C[2]		
模块 3	A[3]	B[3]			C[3]		
模块 4	A[4]	B[4]			C[4]		
模块 5	A[5]	B[5]			C[5]		
模块 6	A[6]	B[6]			C[6]		
模块 7	A[7]	B[7]			C[7]		

4.2.1 “存储器-存储器”结构

第 38 页

解决访问冲突的方法之二：在运算流水线的输入端和输出端增加操作数**可变缓冲器**和写结果**可变缓冲器**，以便消除征用存储器的现象



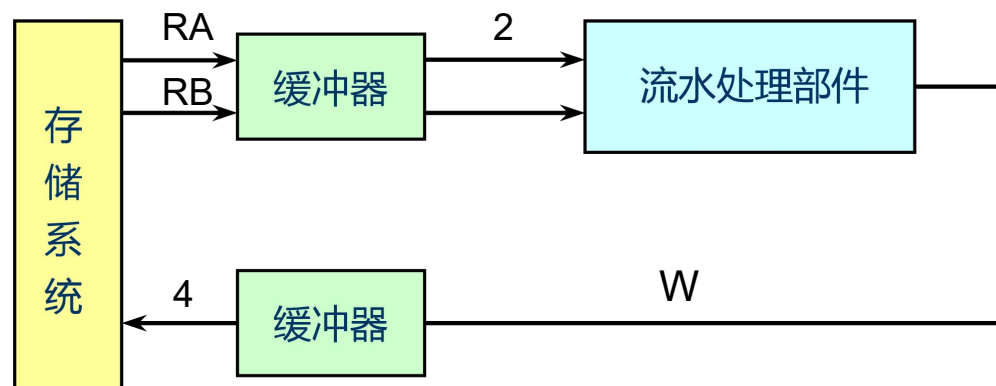
方法：向量A的输入缓冲器延迟**2**个时间周期；输出缓冲器延迟**4**个时间周期

模块 0	A[0]		B[6]			C[4]	
模块 1	A[1]		B[7]			C[5]	
模块 2	A[2]	B[0]				C[6]	
模块 3	A[3]	B[1]				C[7]	
模块 4	A[4]	B[2]			C[0]		
模块 5	A[5]	B[3]			C[1]		
模块 6	A[6]	B[4]			C[2]		
模块 7	A[7]	B[5]			C[3]		

4.2.1 “存储器-存储器”结构

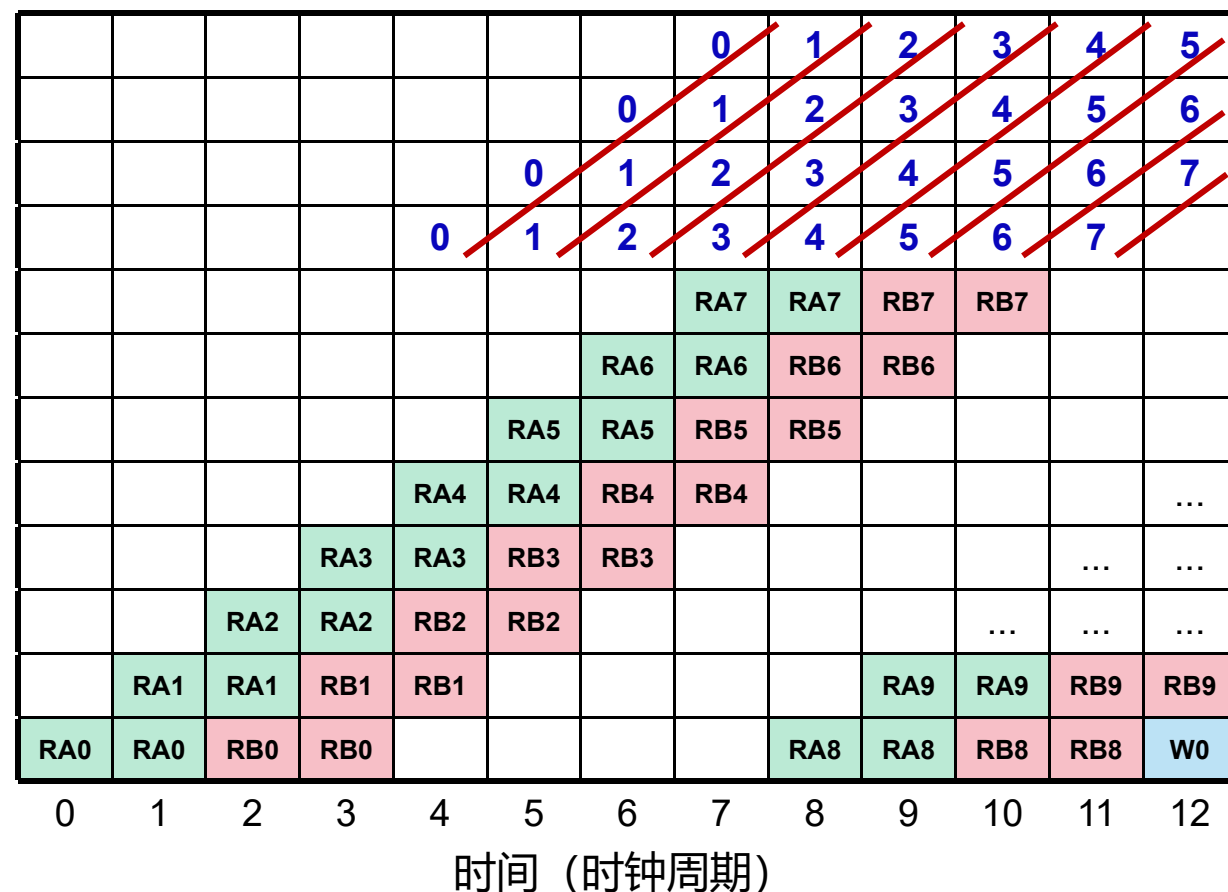
第 39 页

解决访问冲突的方法之二：在运算流水线的输入端和输出端增加操作数**可变缓冲器**和写结果**可变缓冲器**，以便消除征用存储器的现象



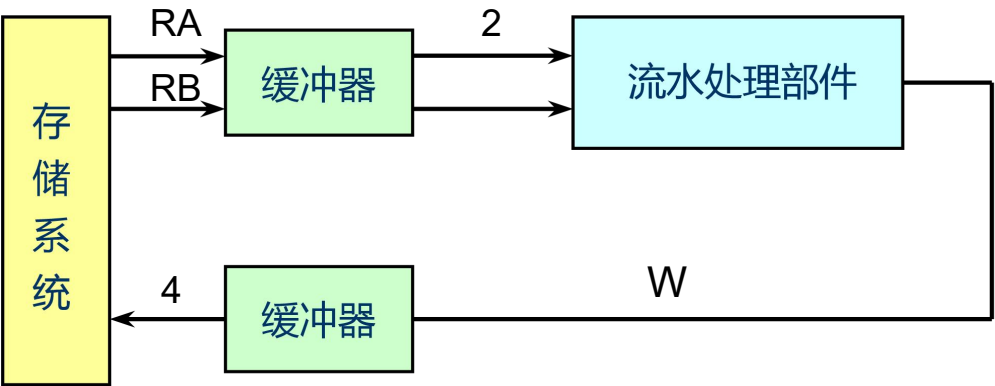
模块 0	A[0]	B[0]			C[0]	
模块 1	A[1]	B[1]			C[1]	
模块 2	A[2]	B[2]			C[2]	
模块 3	A[3]	B[3]			C[3]	
模块 4	A[4]	B[4]			C[4]	
模块 5	A[5]	B[5]			C[5]	
模块 6	A[6]	B[6]			C[6]	
模块 7	A[7]	B[7]			C[7]	

流水线3
流水线3
流水线2
流水线1
存储器7
存储器6
存储器5
存储器4
存储器3
存储器2
存储器1
存储器0



4.2.1 “存储器-存储器”结构

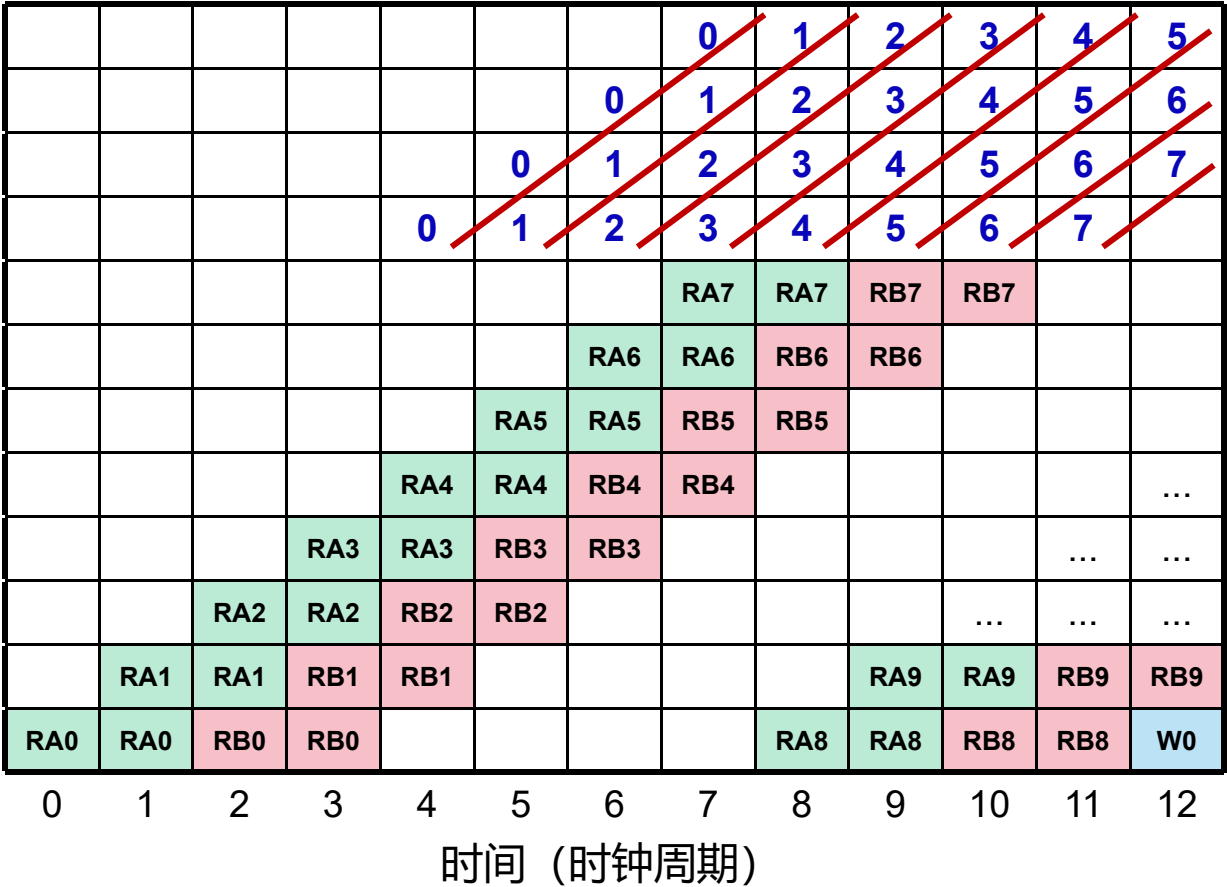
解决访问冲突的方法之二：在运算流水线的输入端和输出端增加操作数**可变缓冲器**和写结果**可变缓冲器**，以便消除征用存储器的现象



方法： 向量A的输入缓冲器延迟**2**个时间周期；输出缓冲器延迟**4**个时间周期

结果： 向量A和B的对应元素同时到达流水结构加法器；输出缓冲器将每个输出值延迟**4**个时间周期再送入存储器系统，第1个结果在时钟周期**12**时写入主存储器。

流水线3
流水线3
流水线2
流水线1
存储器7
存储器6
存储器5
存储器4
存储器3
存储器2
存储器1
存储器0

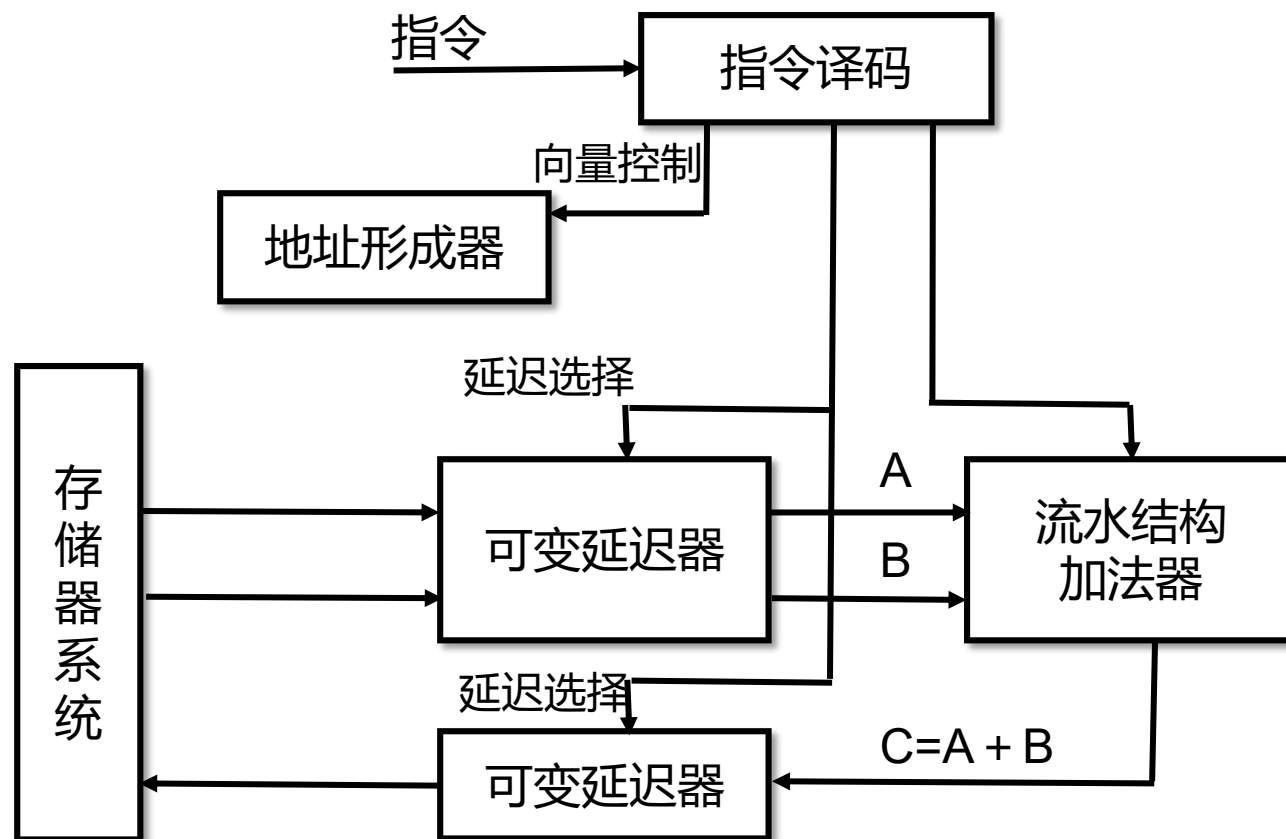


4.2.1 “存储器-存储器”结构

第 41 页

例如，70年代初问世的Star 100

- 存储器：32个体交叉
- 每个体的数据宽度：8个字
(字长64位)
- 最大数据流量：每秒2亿字
- 在运算器的输入和输出端加缓冲器以消除争用存储器现象的方法，本质上与在流水线内部加缓冲器以消除内部冲突的思想是一致的

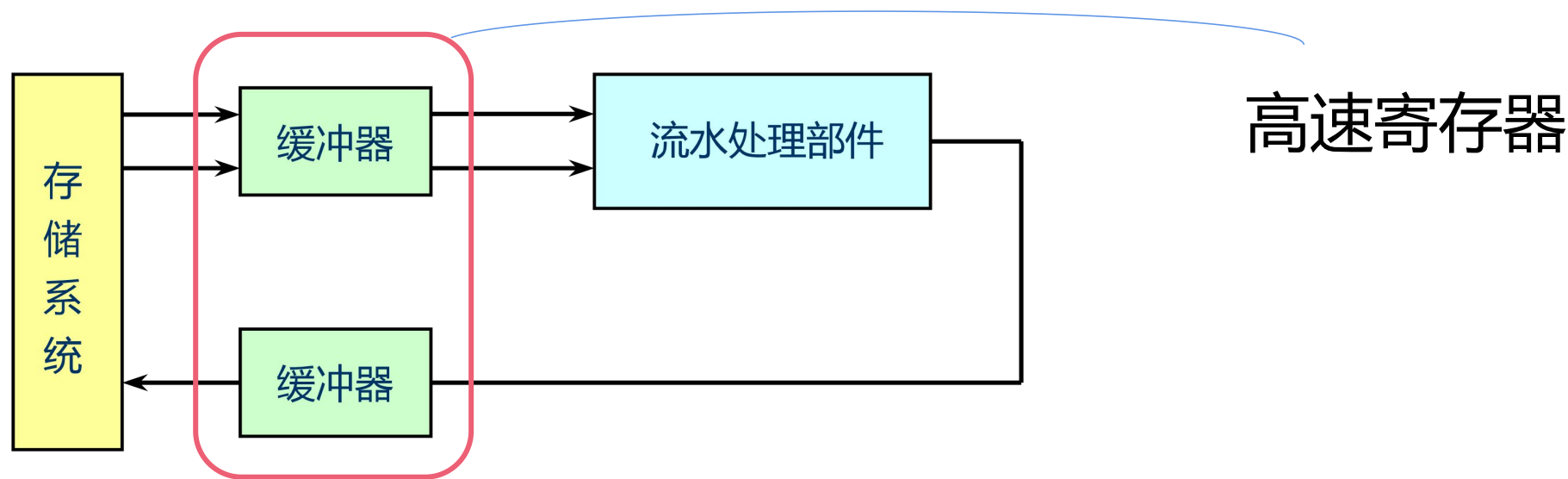


4.2.1 “存储器-存储器”结构

第 42 页

存储器—存储器结构存在的问题：要充分发挥这种结构的流水线效率，存储器要不断地提供源操作数，并不断地从运算部件接收结果，对存储器的带宽以及存储器与处理部件的通信带宽提出了非常高的要求。

解决方案：构造一个具有所要求带宽的高速中间存储器，并能实现该高速中间存储器与主存储器之间的快速数据交换，即寄存器—寄存器结构。



4.2.2 “寄存器-寄存器”结构

第 43 页

■ 采用分组处理方式的向量处理机对处理机结构的要求：**寄存器 - 寄存器结构**

- 把存储器-存储器结构中的缓冲改为向量寄存器。运算部件需要的操作数从向量寄存器中读取，运算的中间结果也写回向量寄存器
- 向量寄存器与标量寄存器的主要区别：
 - ✓ 一个向量寄存器能够保存一个向量，例如64个64位寄存器，能够连续访问一向量的各个分量。
- 优点：降低主存储器的流量

4.2.2 “寄存器-寄存器”结构

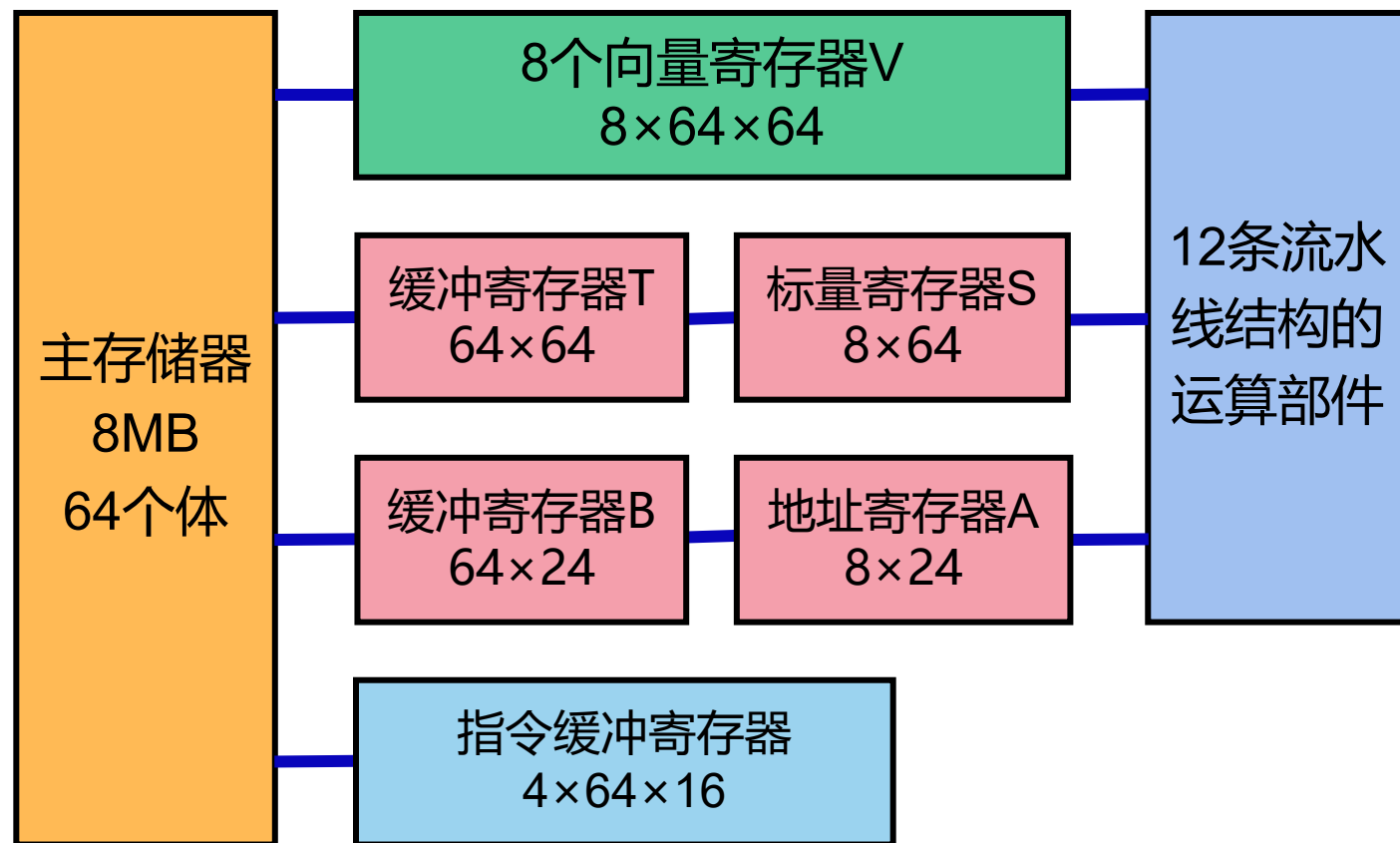
第 44 页

■典型的寄存器 - 寄存器结构的向量处理机：美国的CRAY-1、我国的YH-1巨型机等

■以CRAY-1机为例

- 美国CRAY公司
- 1976年
- 每秒1亿次浮点运算
- 时钟周期：12.5ns

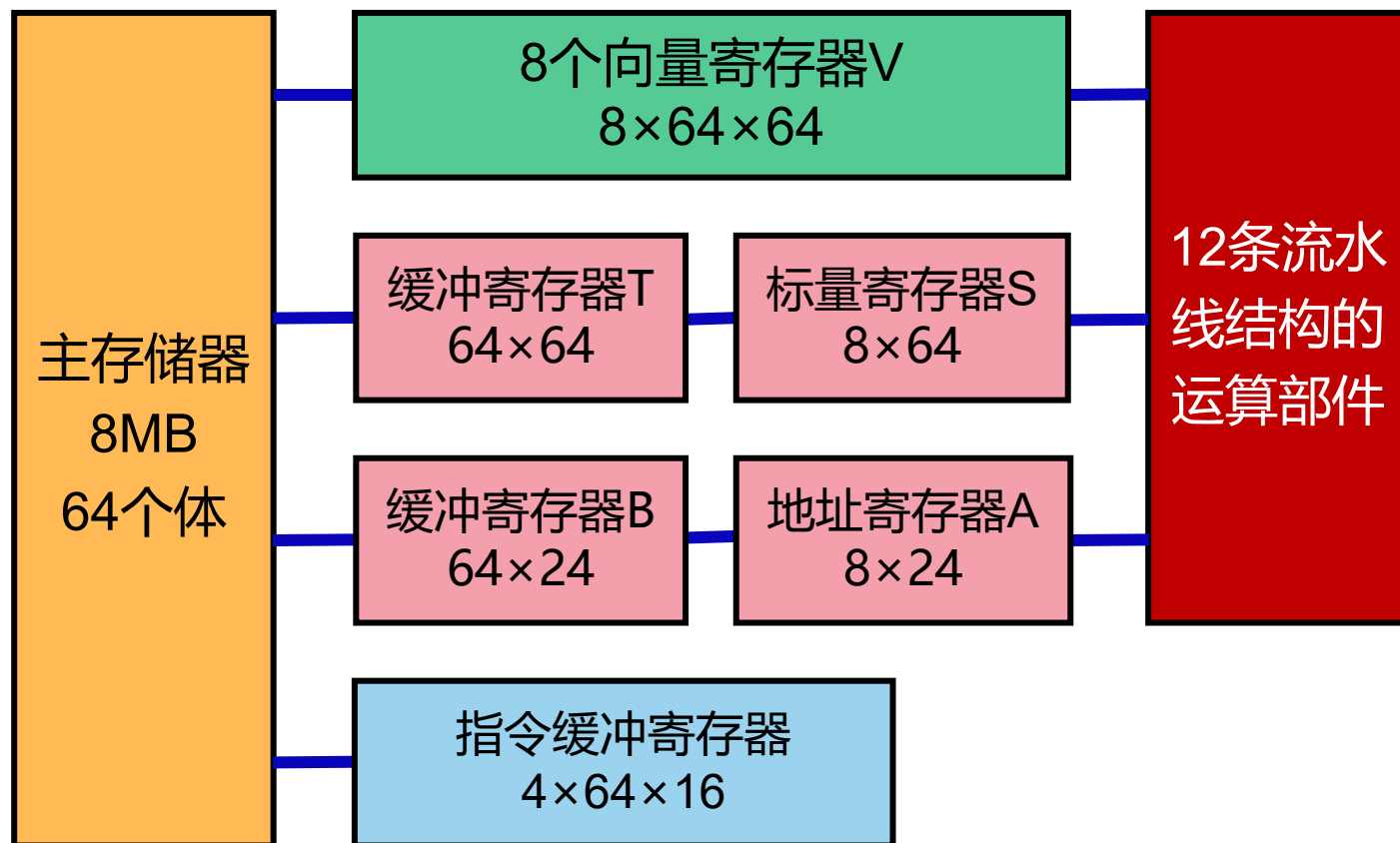
与存储器-存储器结构的STAR-100比较，
运算速度3倍多（时钟周期为40:12.5），
主存储器流量低2.5倍。



4.2.2 “寄存器-寄存器”结构

第 45 页

1. 基于分级存储系统的CRAY-1向量处理结构



➤ 功能部件

共有12条可并行工作的单功能流水线，可分别流水地进行地址、向量、标量的各种运算

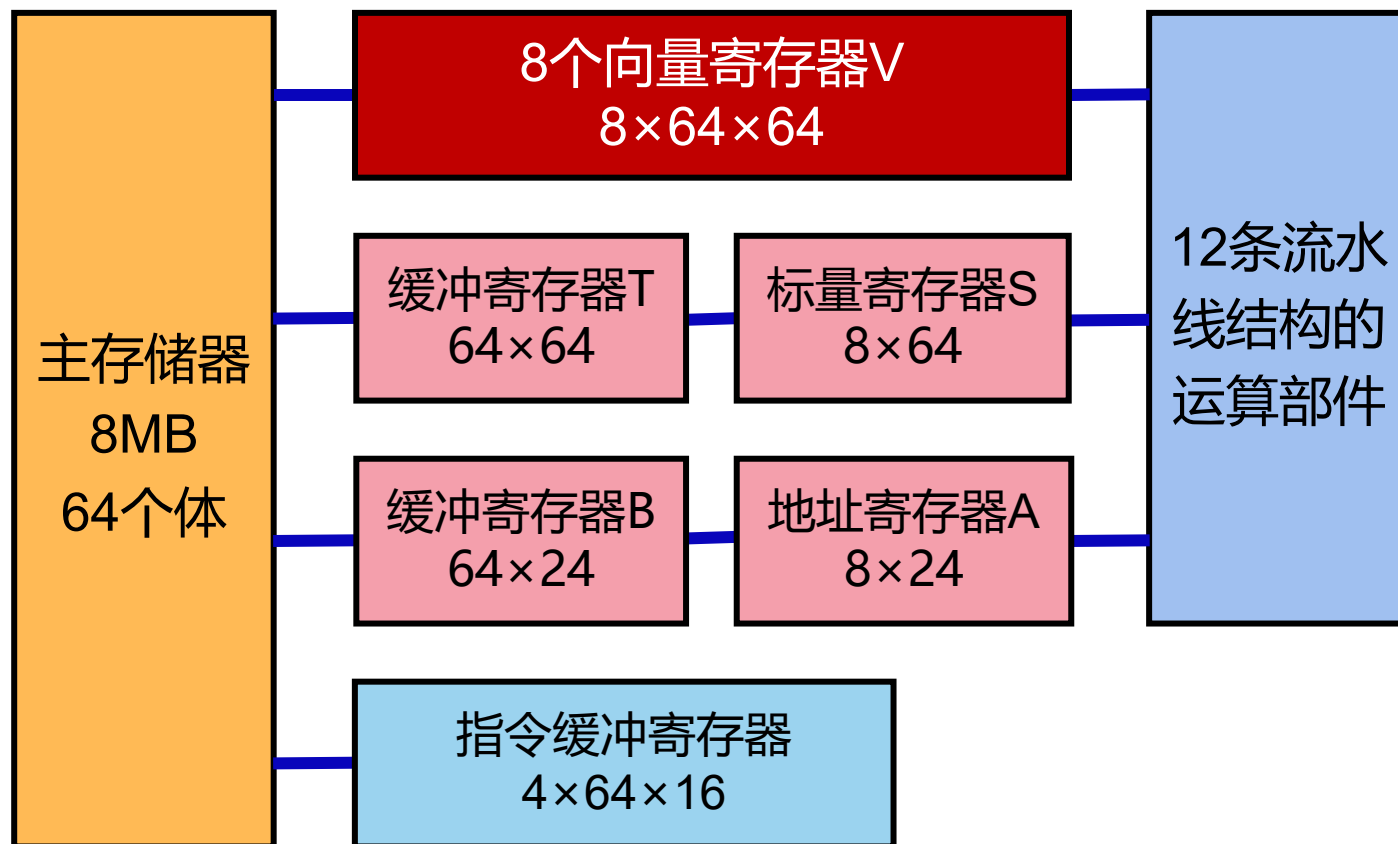
- ✓ 整数加 (3拍)
- ✓ 逻辑运算 (2拍)
- ✓ 移位 (4拍)
- ✓ 浮点加 (6拍)
- ✓ 浮点乘 (7拍)
- ✓ 浮点迭代求倒数 (14拍)

括号中的数字为其流水经过的时间，每拍为一个时钟周期，即12.5ns。

4.2.2 “寄存器-寄存器”结构

第 46 页

1. 基于分级存储系统的CRAY-1向量处理结构

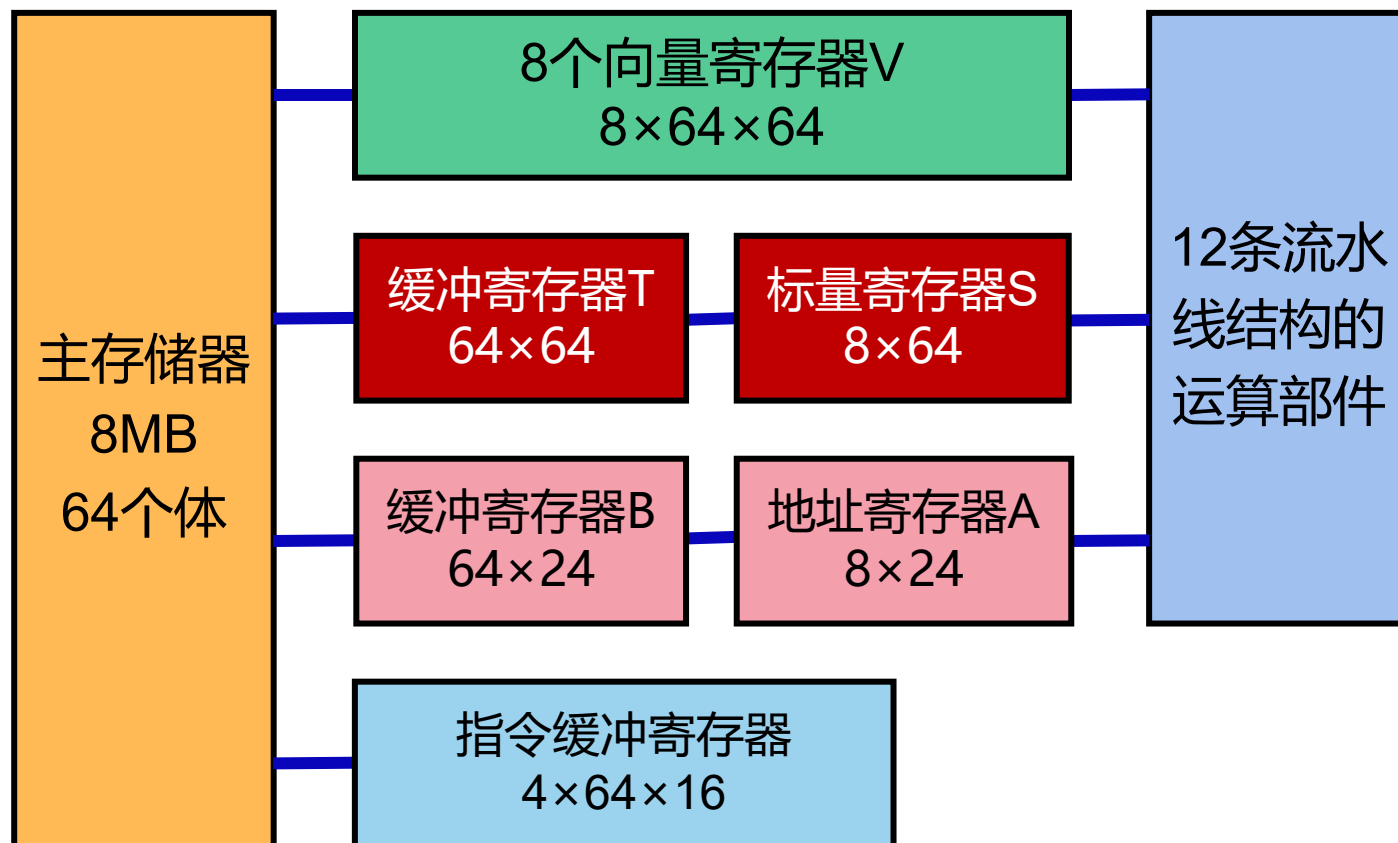


- 对于向量运算来说，中间存储器是V寄存器，它是向量寄存器，由8个64个分量的寄存器组成，每个分量为一个64位寄存器
- 流水结构运算器在一个时钟周期内从两个V寄存器得到一对操作数，完成某种操作后用一个时钟周期的时间把结果送入另一个V寄存器
- 主存储器与V寄存器之间的数据传送的方式进行

4.2.2 “寄存器-寄存器”结构

第 47 页

1. 基于分级存储系统的CRAY-1向量处理结构



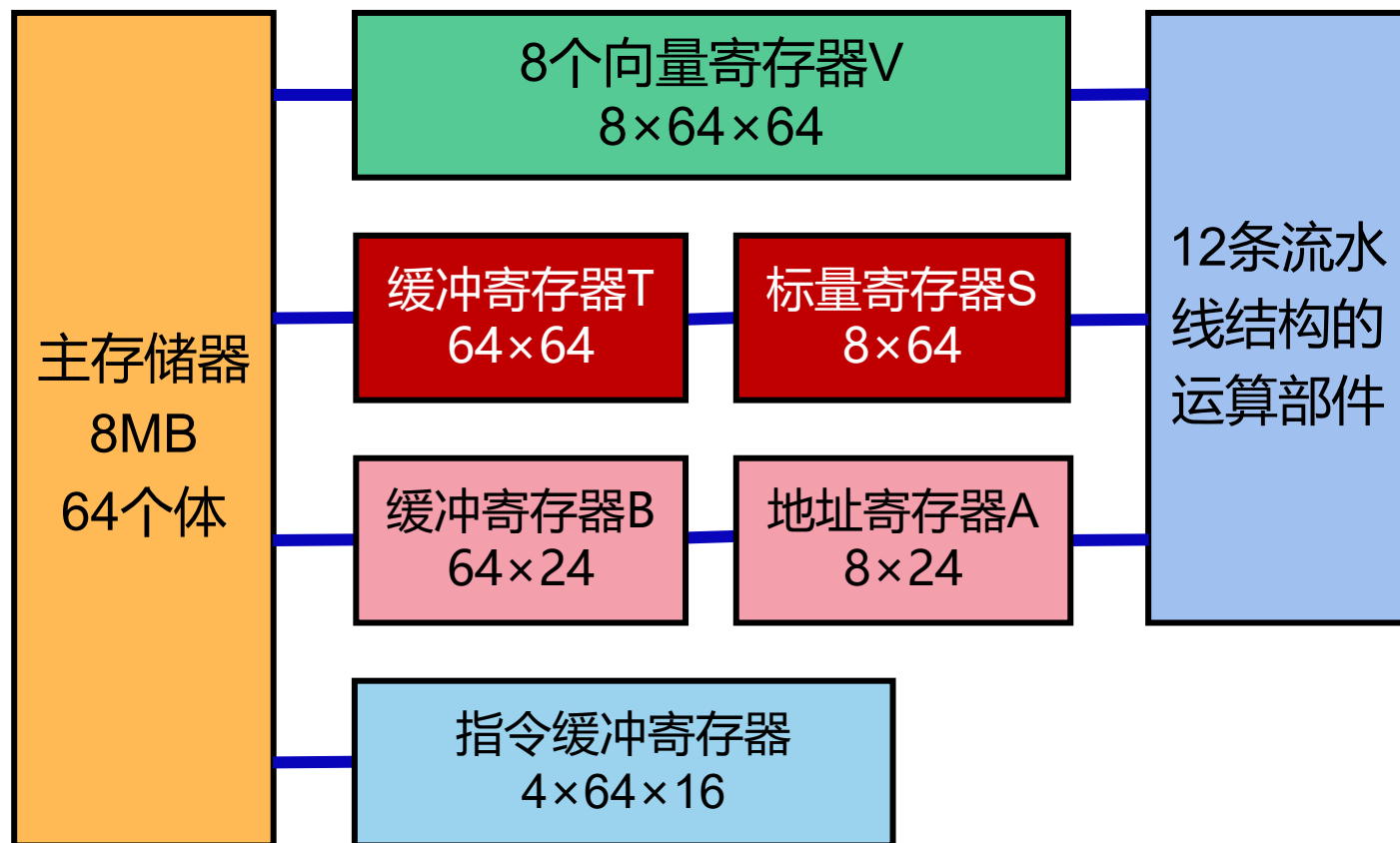
对于标量运算而言，有两级中间存储器：

- S寄存器：速度很快的一级，是8个64位的为标量运算和逻辑运算提供源寄存器和结果寄存器
- T寄存器：速度稍慢但仍很高速的中间寄存器，由64个标量寄存器组成，每个寄存器字长64位。
- 主存储器与T寄存器之间以成组传送的方式进行数据传送

4.2.2 “寄存器-寄存器”结构

第 48 页

1. 基于分级存储系统的CRAY-1向量处理结构

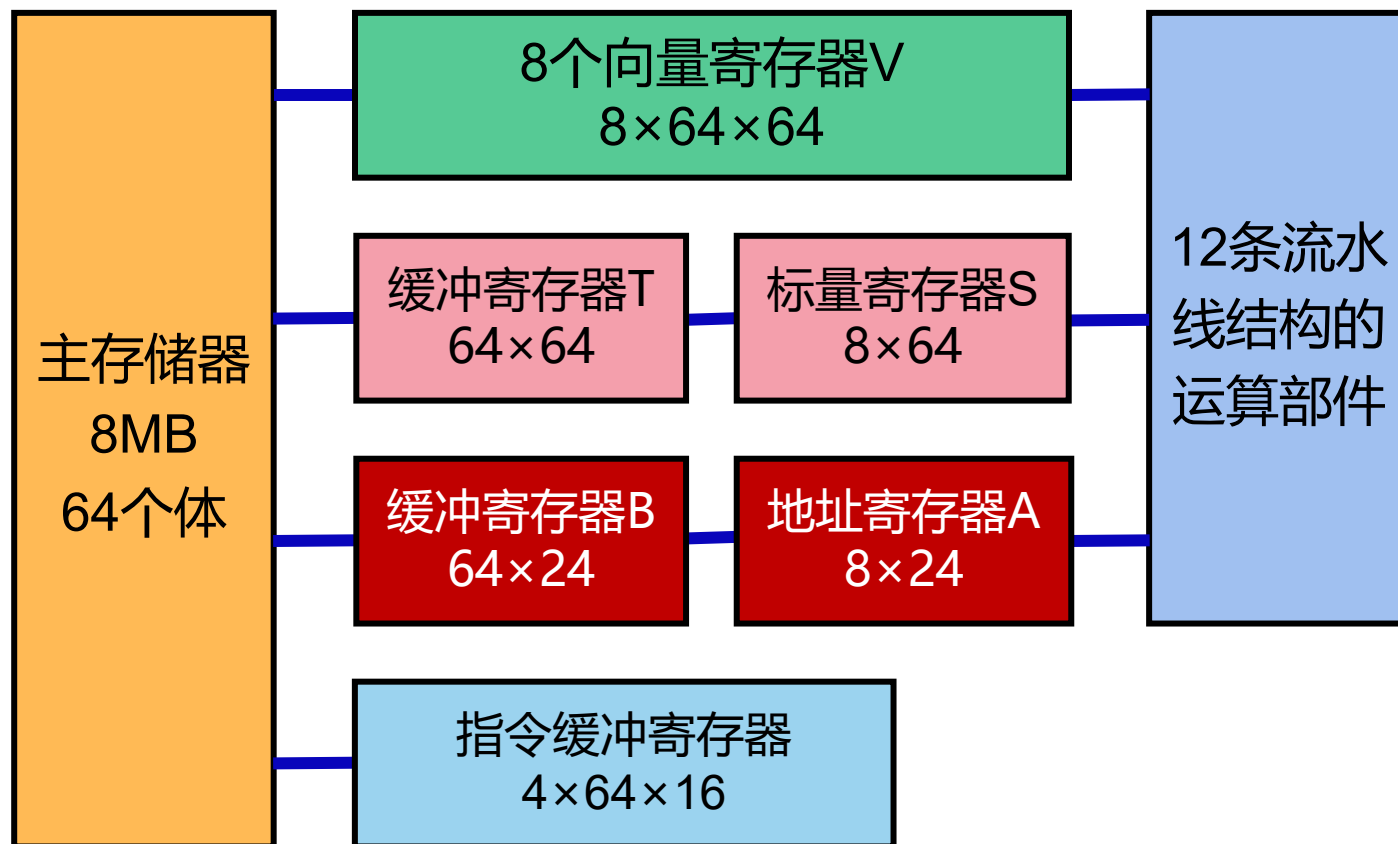


- T寄存器与CACHE存储器相同，都是为了保存那些高速的标量存储器装不下的数据
- 与CACHE存储器不同的是这些中间存储器不是自动管理的，而是由程序员或编译程序来管理（编程可获得更高的性能）

4.2.2 “寄存器-寄存器”结构

第 49 页

1. 基于分级存储系统的CRAY-1向量处理结构

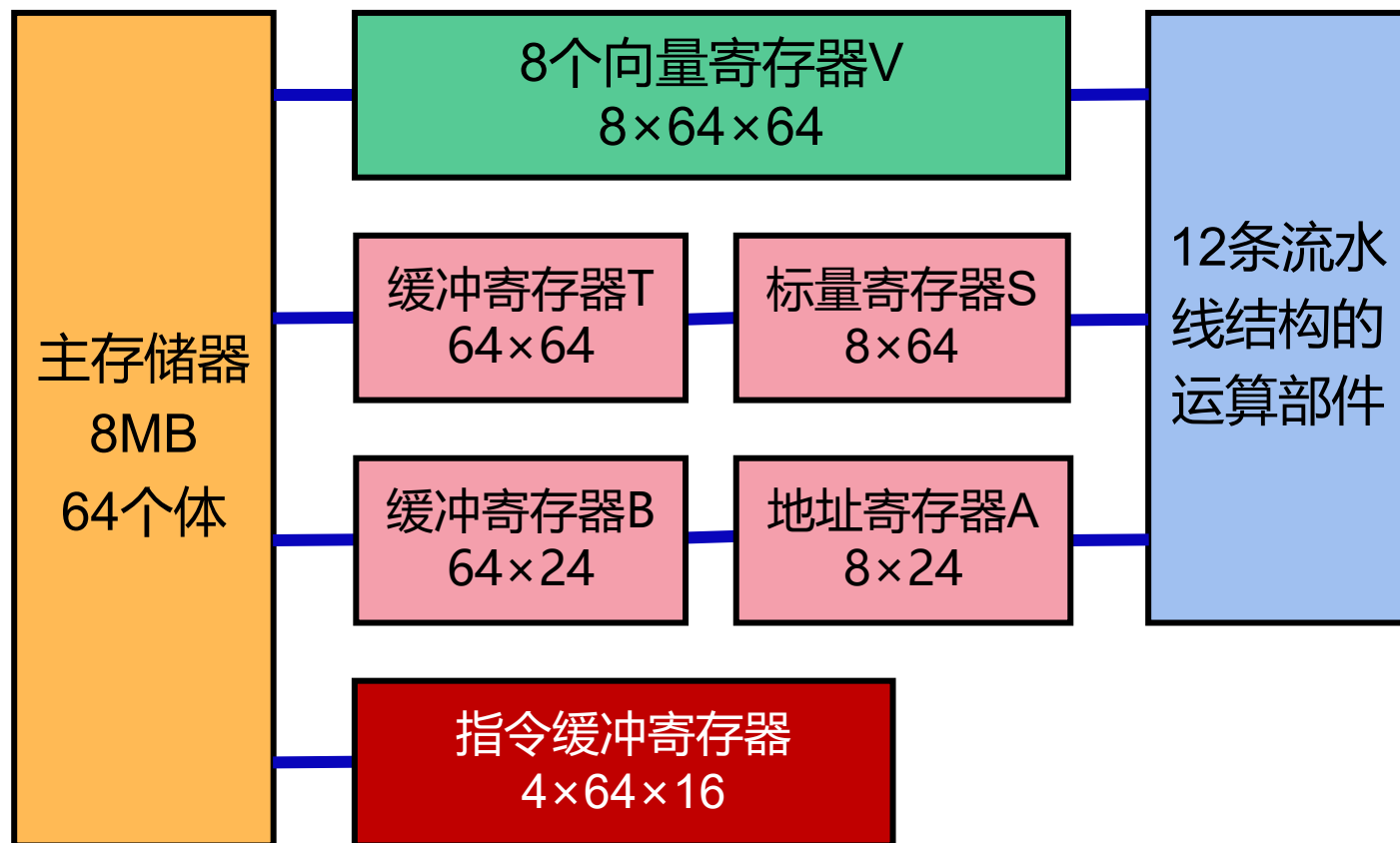


- 8个64位的A寄存器，主要用作访问存储器的地址寄存器和变址寄存器，还可用来提供移位的计算值和循环控制值
- B寄存器用作A寄存器的中间存储器，可以存放需要重复访问的数据

4.2.2 “寄存器-寄存器”结构

第 50 页

1. 基于分级存储系统的CRAY-1向量处理结构

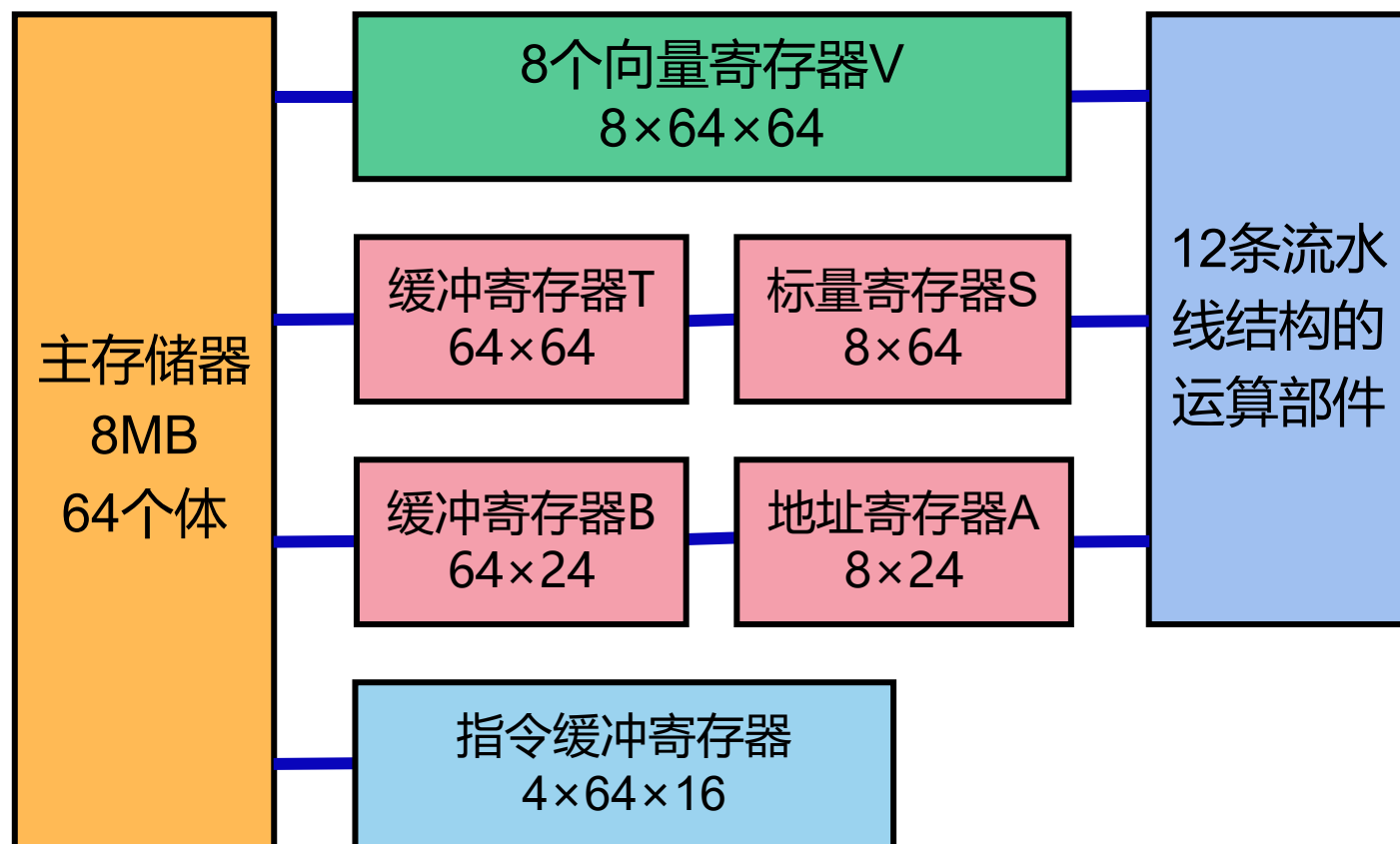


- 指令缓冲器，它由256个16位寄存器组成，用来存放在指令执行之前就预取出来的指令
- 流水结构运算器的每个功能部件都有一个高速存储器与之相连，没有一个功能部件是直接与主存相连。

4.2.2 “寄存器-寄存器”结构

第 51 页

1. 基于分级存储系统的CRAY-1向量处理结构



这种结构设计的主要思想:

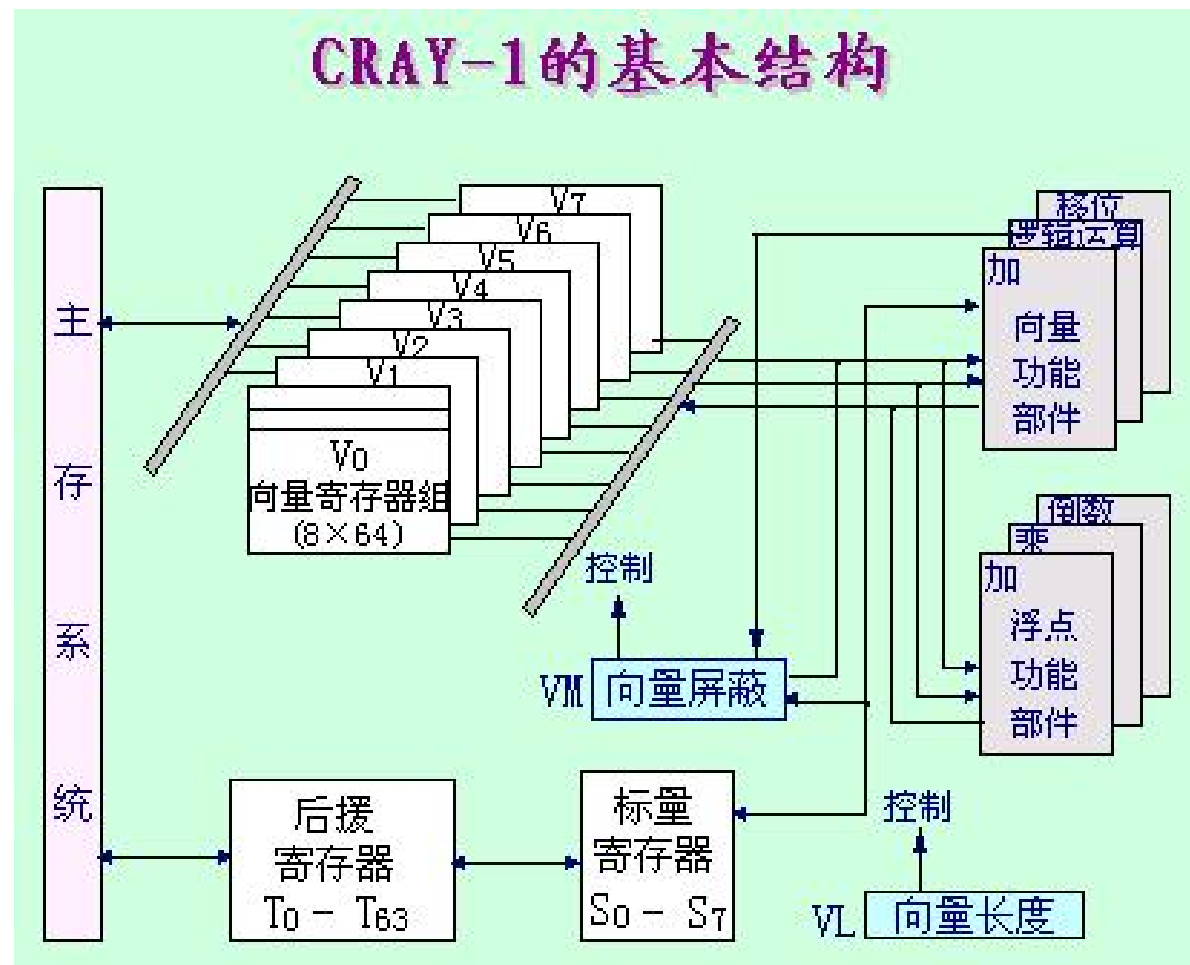
- 流水结构运算器的每个功能部件都有一个高速存储器与之相连, **没有一个功能部件是直接**与主存相连;
- 使操作数离处理器很近, 以保证处理器一起处于忙状态;
- 中间存储器能提供处理器快速存取的数据。

4.2.2 “寄存器-寄存器”结构

第 52 页

2. CRAY-1向量处理的显著特点

- 每个向量寄存器 V_i 都有连到6个向量功能部件的单独总线。每个向量功能部件也都有把运算结果送回向量寄存器组的总线。
- 只要不出现 V_i 冲突和功能部件冲突, 各 V_i 之间和各功能部件之间都能并行工作, 大大加快了向量指令的处理

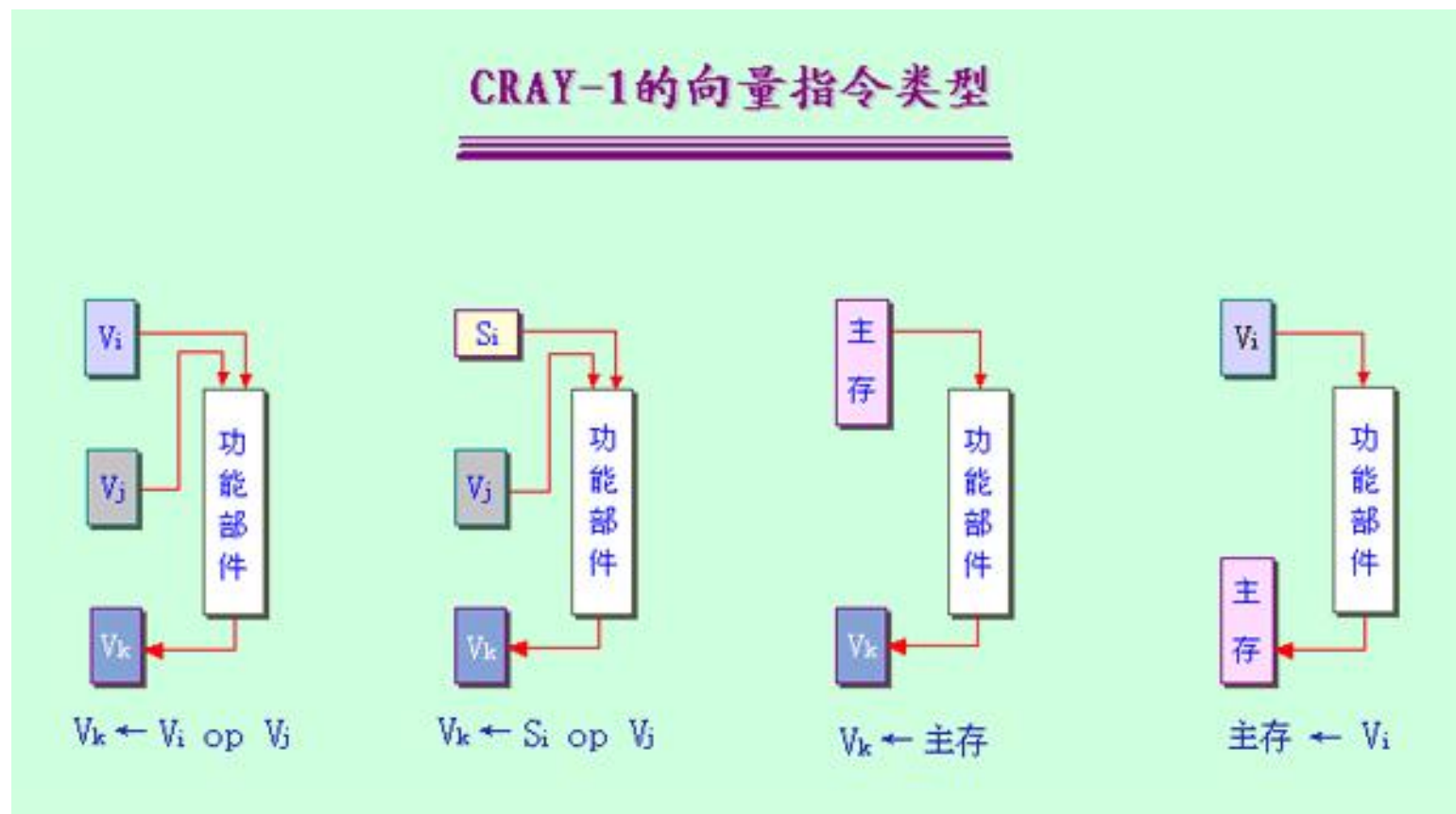


4.2.2 “寄存器-寄存器”结构

第 53 页

3. CRAY-1向量指令类型

- ① $V_k \leftarrow V_i \text{ op } V_j$
- ② $V_k \leftarrow S_i \text{ op } V_j$
- ③ $V_k \leftarrow \text{主存}$
- ④ $\text{主存} \leftarrow V_i$



4.3 提高向量处理机性能的常用技术

提高向量处理机性能的方法

- 设置多个功能部件，使它们并行工作；
- 采用链接技术，加快一串向量指令的执行；
- 采用分段开采技术，加快循环的处理；
- 采用多处理机系统，进一步提高性能。

4.3.1 设置多个功能部件

第 56 页

1. 设置多个独立的功能部件。这些部件能并行工作，并各自按流水方式工作，从而形成了多条并行工作的运算操作流水线。

例如CRAY-1向量处理机有4组12个单功能流水部件：

- 向量部件：向量加，移位，逻辑运算
- 浮点部件：浮点加，浮点乘，浮点求倒数
- 标量部件：标量加，移位，逻辑运算，数“1”/计数
- 地址运算部件：整数加，整数乘

2. 向量流水线冲突分析

■设计目标：保证处理器一起处于忙状态，导致无法满负荷工作的原因是向量流水线冲突。

■向量流水线冲突分析：两条向量指令占用功能流水线和向量寄存器的4种情况：

① 指令不相关

例如： $V0 \leftarrow V1 + V2$

$V6 \leftarrow V4 * V5$

这两条指令分别使用各自所需的流水线和向量寄存器，可以并行执行。

② 功能部件冲突

例如： $V3 \leftarrow V1 + V2$

$V6 \leftarrow V4 + V5$

这两条指令都要使用加法流水线，发生了功能部件冲突（但向量寄存器不冲突）。当第一条指令流出时，占用加法流水线。第二条指令要等加法流水线变成空闲后，才能流出。

③ 源寄存器冲突

例如： $V3 \leftarrow V1 + V2$

$V6 \leftarrow V1 * V4$

这两条向量指令的源向量之一都取自V1。由于两者的首元素下标可能不同，向量长度也可能不同，所以难以由V1同时提供两条指令所需要的源向量。这两条向量指令不能同时执行。只有等第一条向量指令执行完、释放V1之后，第二条向量指令才能开始执行。

4.3.2 链接技术

第 59 页

④ 结果寄存器冲突——两条向量指令使用了相同的结果向量寄存器。

例如： $V4 \leftarrow V1 + V2$

$V4 \leftarrow V3 * V5$

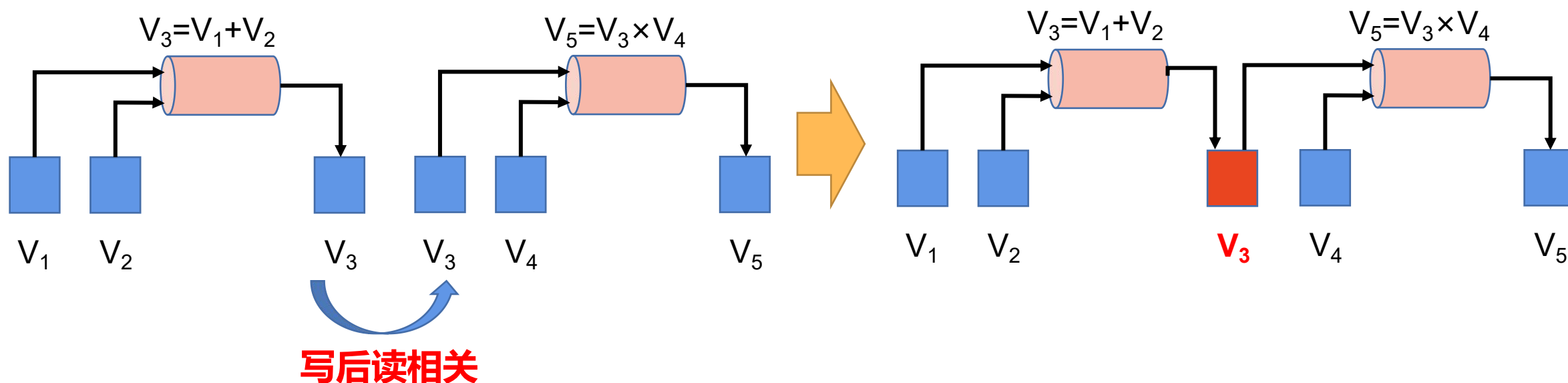
这两条指令都要访问目的寄存器V4。由于第一条指令在先，所以它先占用V4直到运算完成，然后再流出后一条指令。

4.3.2 链接技术

第 60 页

3. 向量流水线链接技术：具有先写后读相关的两条指令，在不出现功能部件冲突和源向量冲突的情况下，可以把功能部件链接起来进行流水处理，以达到加快执行的目的。

■前一条指令的结果不必送回存储器，①直接作为后一条指令的操作数

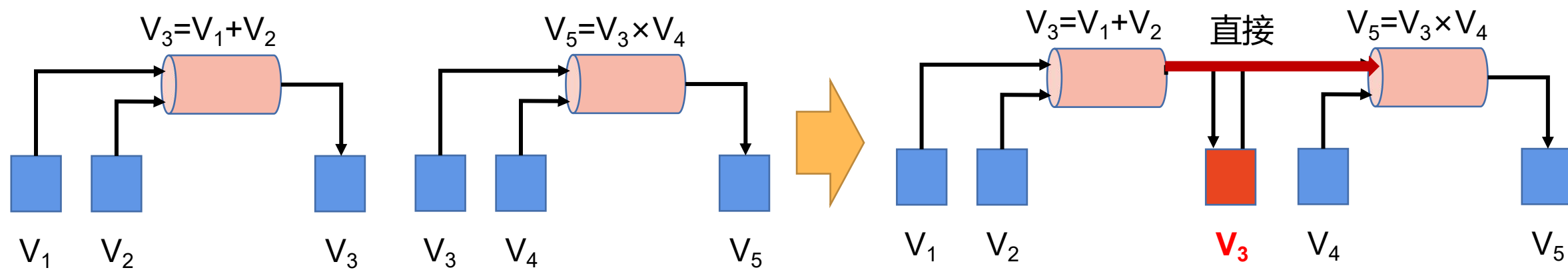


4.3.2 链接技术

第 61 页

3. 向量流水线链接技术:

- 前一条指令的结果不必送回存储器，②甚至可在前一条指令完成之前就使用其结果。



结果寄存器可能成为后继指令的操作数寄存器，在CRAY-1中这种技术被称为两条流水线的链接 (chaining)

4.3.2 链接技术

第 62 页

例如，当前一条指令的结果寄存器是后一条指令的源寄存器、且不存在任何其他冲突时，就可以用链接技术来提高性能。

$$V3 \leftarrow V1 + V2$$

$$V6 \leftarrow V3 * V4$$

- Cray-1 向量处理的一个显著特点
- 链接特性的实质——把流水线定向的思想引入到向量执行过程的结果
- 链接时，Cray-1 中把向量数据元素送往向量功能部件以及把结果存入向量寄存器都需要一拍时间，从存储器中把数据送入访存功能部件也需要一拍时间（同步的要求）。

4.3.2 链接技术

第 63 页

例4.1 考虑在Cray-1上利用链接技术执行以下4条指令：

$V_0 \leftarrow \text{存储器}$ // 访存取向量：7拍

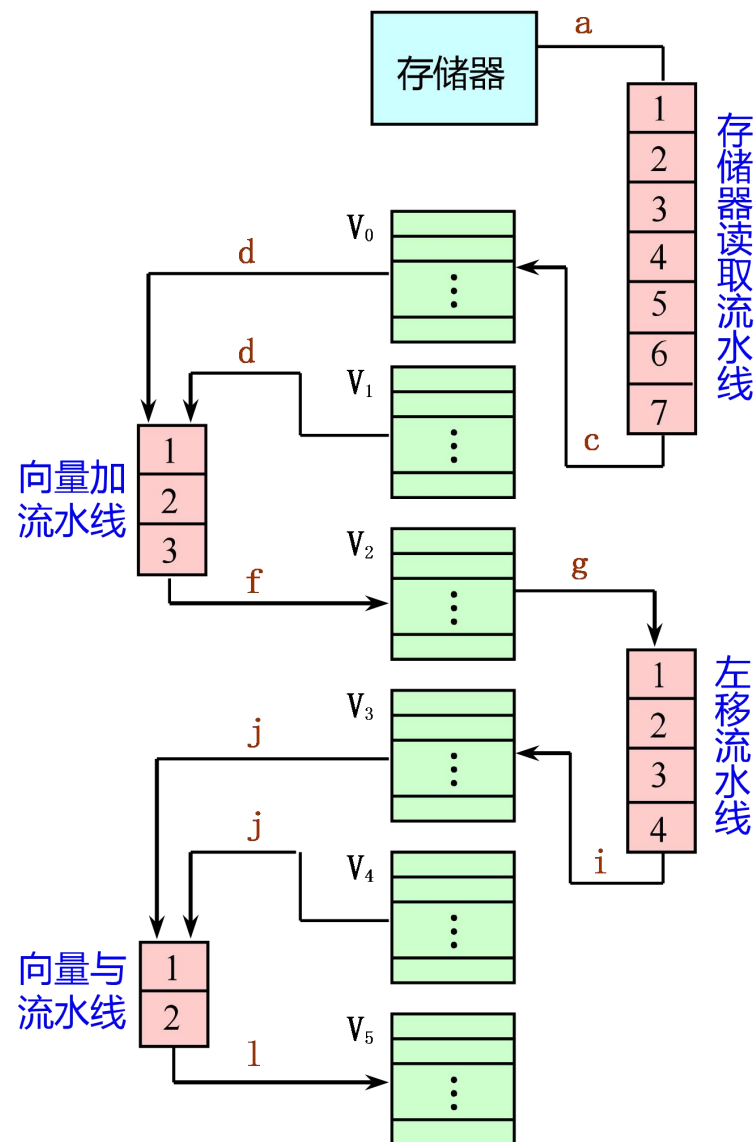
$V_2 \leftarrow V_0 + V_1$ // 向量加：3拍

$V_3 \leftarrow V_2 \ll A_3$ // 按 (A_3) 左移：4拍

$V_5 \leftarrow V_3 \wedge V_4$ // 与操作：2拍

画出链接示意图，并求该链接流水线的通过时间。

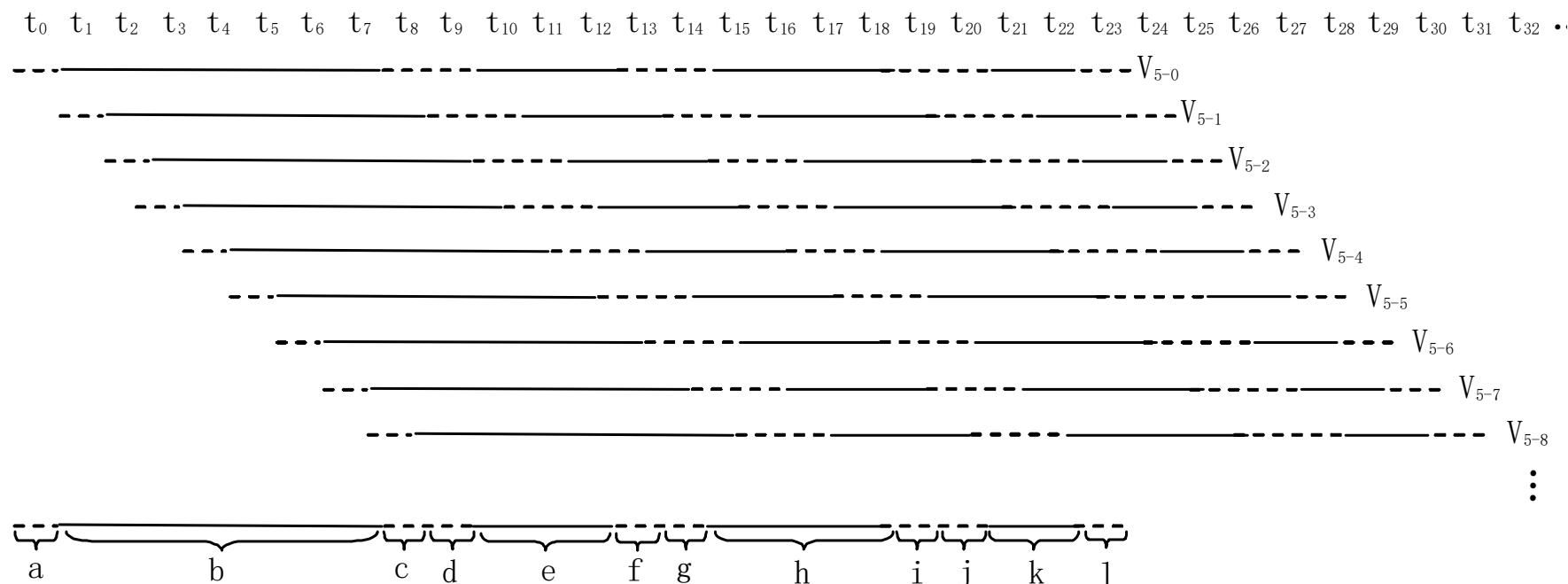
解：对这4条指令进行分析可知：它们既没有部件冲突，也没有寄存器冲突，相邻两条指令之间都存在先写后读相关，因而可以把访存流水线、向量加流水线、向量移位流水线以及向量逻辑运算流水线链接成一个较长的流水线。



4.3.2 链接技术

第 64 页

链接操作的时间图：



a: 存储字到“读功能部件”的传送时间

b: 存储字经过“读功能部件”的通过时间

c: 存储字从“读功能部件”到 V_0 分量的传送时间

d: V_0 和 V_1 中操作数到整数加功能部件的传送时间

e: 整数加功能部件的通过时间

f: 和从整数加功能部件到 V_2 分量的传送时间

g: V_2 中的操作数分量到移位功能部件的传送时间

h: 移位功能部件的通过时间

i: 结果从移位功能部件到 V_3 分量的传送时间

j: V_3 和 V_4 中的操作数分量到逻辑部件的传送时间

k: 逻辑功能部件的通过时间

l: 最后结果到 V_5 分量的传送时间

4.3.2 链接技术

第 65 页

例4.2 在CRAY-1上用链接技术进行向量运算

$$D = A \times (B + C)$$

假设向量长度 $N \leq 64$ ，向量元素为浮点数，且向量B、C已存放在 V_0 和 V_1 中。

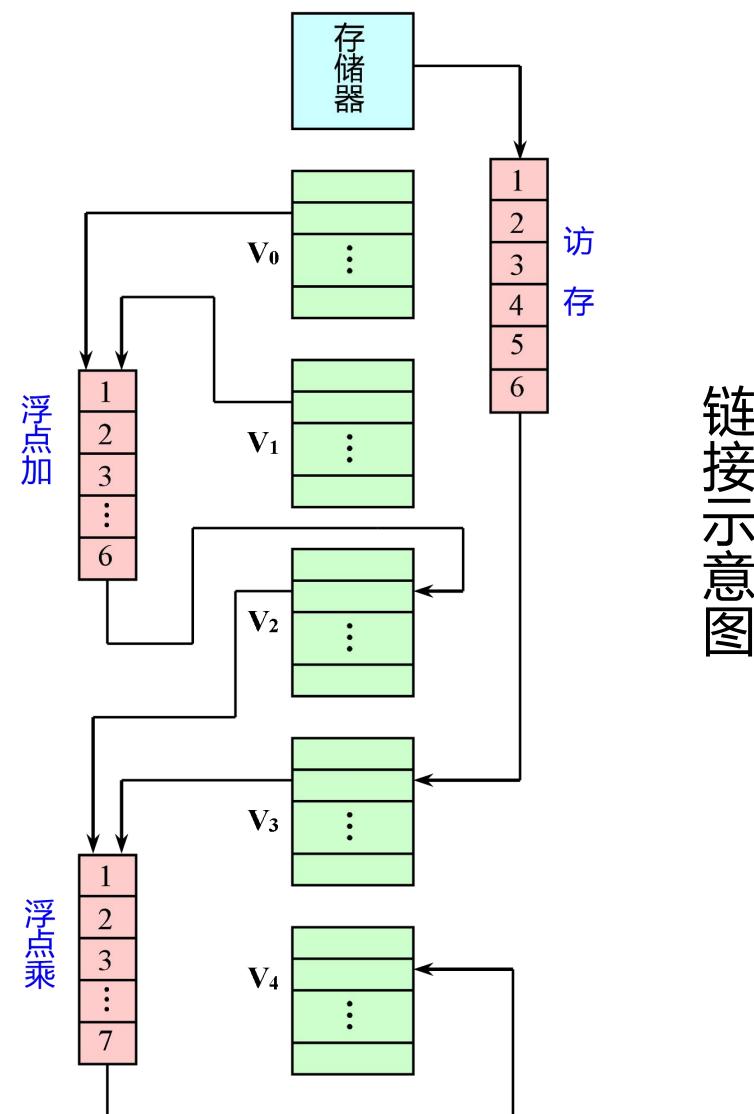
画出链接示意图，并分析非链接执行和链接执行两种情况下的执行时间。

解 用以下三条向量完成上述运算：

$V_3 \leftarrow \text{存储器}$ // 访存取向量A

$V_2 \leftarrow V_0 + V_1$ // 向量B和向量C进行浮点加

$V_4 \leftarrow V_2 \times V_3$ // 浮点乘，结果存入 V_4



4.3.2 链接技术

第 66 页

- 3条指令全部用串行方法执行，则执行时间为：

$$[(1 + 6 + 1) + N - 1] + [(1 + 6 + 1) + N - 1] + [(1 + 7 + 1) + N - 1] = 3N + 22 \quad (\text{拍})$$

- 前两条指令并行执行，然后再串行执行第3条指令，则执行时间为：

$$[(1 + 6 + 1) + N - 1] + [(1 + 7 + 1) + N - 1] = 2N + 15 \quad (\text{拍})$$

- 第1、2条向量指令并行执行，并与第3条指令链接执行。

$$[(1 + 6 + 1)] + [(1 + 7 + 1)] + (N - 1) = N + 16 \quad (\text{拍})$$

- 进行向量链接的要求，保证：无向量寄存器使用冲突和无功能部件使用冲突

- 只有在前一条指令的第一个结果元素送入结果向量寄存器的那一个时钟周期才可以进行链接。
- 当一条向量指令的两个源操作数分别是两条先行指令的结果寄存器时，要求先行的两条指令产生运算结果的时间必须相等，即要求有关功能部件的通过时间相等。
- 要进行链接执行的向量指令的向量长度必须相等，否则无法进行链接。

4.3.3 分段开采技术

第 67 页

如果向量的长度大于向量寄存器的长度，该如何处理呢？

- 当向量的长度大于向量寄存器的长度时，必须把长向量分成长度固定的段，然后循环分段处理，每一次循环只处理一个向量段。
- 这种技术称为分段开采技术。
 - 由系统硬件和软件控制完成，对程序员是透明的。

4.3.3 分段开采技术

第 68 页

例4.3 设计A和B是长度为N的向量，考虑在CRAY—1 向量处理机上实现如下的循环操作

```
for(i=1;i<=N;i++)  
    A[i] = 5.0 * B[i] + 1.0
```

情况1：当 $N \leq 64$ 时，可以用如下指令序列实现上述循环操作：

$S_1 \leftarrow 5.0$	;将常数5.0送往标量寄存 S_1
$S_2 \leftarrow 1.0$	;将常数1.0送往标量寄存 S_2
$VL \leftarrow N$	;将向量寄存器VL中设置向量长度N
$V_0 \leftarrow B$	;从存储器中将向量B读入向量寄存器 V_0
$V_1 \leftarrow S_1 * V_0$	;向量B中的每个元素分别和常数 S_1 相乘
$V_2 \leftarrow S_2 + V_1$	;向量 V_1 中的每个元素分别和常数 S_2 相加
$A \leftarrow V_2$	;将计算结果从向量寄存器 V_2 存入存储器的向量A

4.3.3 分段开采技术

第 69 页

例4.3 设计A和B是长度为N的向量，考虑在CRAY—1 向量处理机上实现如下的循环操作

```
for(i=1;i<=N;i++)  
    A[i] = 5.0 * B[i] + 1.0
```

情况2：当 $N > 64$ 时，就需要进行分段开采计算，首先根据向量长度计算出循环次数K，设 $N=1000$ ，则

$$K = \left\lfloor \frac{N}{64} \right\rfloor = 15 \qquad 1000 \div 64 = 15.625$$

一般来说，N不一定总是64的倍数，所以如果 $N \div 64$ 有余数，则余数L为：

$$L = N - \left\lfloor \frac{N}{64} \right\rfloor \times 64 = 1000 - 64 \times 15 = 1000 - 960 = 40$$

4.3.3 分段开采技术

第 70 页

例4.3 设计A和B是长度为N的向量，考虑在CRAY—1 向量处理机上实现如下的循环操作

```
for(i=1;i<=N;i++)  
    A[i] = 5.0 * B[i] + 1.0
```

情况2：当 $N > 64$ 时，进行分段开采计算

① 处理余数部分，计算L个元素

```
S1 ← 5.0           ;将常数5.0送往标量寄存S1  
S2 ← 1.0           ;将常数1.0送往标量寄存S2  
VL ← N              ;将向量寄存器VL中设置向量长度N  
V0 ← B              ;从存储器中将向量B读入向量寄存器V0  
V1 ← S1 * V0       ;向量B中的每个元素分别和常数S1相乘  
V2 ← S2 + V1       ;向量V1中的每个元素分别和常数S2相加  
A ← V2              ;将计算结果从向量寄存器V2存入存储器  
的向量A[0,L-1]
```

② 循环K次，分组处理

```
for(i=0 to K-1)  
{  
    V0 ← B            ;从存储器中将向量B读入向量寄存器V0  
    V1 ← S1 * V0     ;向量B中的每个元素分别和常数S1相乘  
    V2 ← S2 + V1     ;向量V1中的每个元素分别和常数S2相加  
    A ← V2            ;将计算结果从向量寄存器V2存入存储器  
的向量A[L+i*64, L+i*64+63]  
}
```

4.3.4 采用多处理机系统

第 71 页

许多新型向量处理机系统采用了多处理机系统结构，例如：

■ CRAY-2

- 包含了4个向量处理机
- 浮点运算速度最高可达1800MFLOPS

■ CRAY Y-MP、C90

- 最多可包含16个向量处理机

4.4 向量处理机的性能评价

4.4 向量处理机的性能评价

第 73 页

衡量向量处理机性能的主要参数：

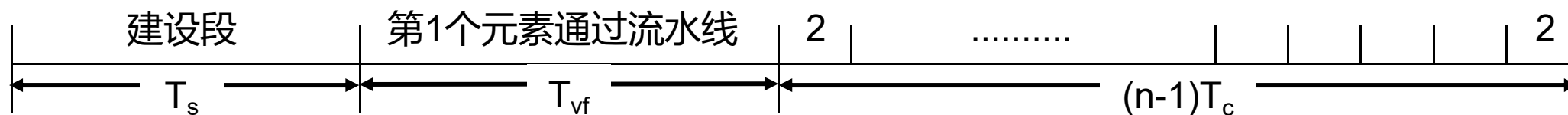
- 向量指令的处理时间
- 向量长度为无穷大时的向量处理机的最大性能
- 半性能向量长度
- 向量长度临界值

4.4.1 向量指令的处理时间 T_{vp}

第 74 页

1. 一条向量指令的处理时间 T_{vp}

假定指令是预取的，一条向量指令的执行时间将由三部分组成，如图所示。



- 建设段 T_s 是为向量指令的执行进行准备的阶段
- 第二段 T_{vf} 是使被处理的向量中第一个元素通过流水线所花费的时间
- 第三段是所有后续元素通过流水线所花费的时间，每个元素只用了一个时间周期。

4.4.1 向量指令的处理时间 T_{vp}

第 75 页

1. 一条向量指令的处理时间 T_{vp}

➤ 执行一条向量长度为 n 的向量指令所需的时间为：

$$T_{vp} = T_s + T_e + (n - 1)T_c$$

✓ T_s ：向量处理部件流水线的建立时间

为了使处理部件流水线能开始工作（即开始流入数据）所需要的准备时间。

✓ T_e ：向量流水线的通过时间

第一对向量元素通过流水线并产生第一个结果所花的时间。

✓ T_c ：流水线的时钟周期时间，每个元素只用了一个时间周期

4.4.1 向量指令的处理时间 T_{vp}

第 76 页

- 把上式中的参数都折算成时钟周期个数：

$$T_{vp} = [s + e + (n - 1)]T_c = [s + e + (n - 1)]\Delta t$$

➤ s ： T_s 所对应的时钟周期数

➤ e ： T_e 所对应的时钟周期数

- 不考虑 T_s ，并令 $T_{start} = e - 1$

$$T_{vp} = (T_{start} + n)T_c$$

- T_{start} ：从一条向量指令开始执行到还差一个时钟周期就产生第一个结果所需的时钟周期数。可称之为该向量指令的启动时间。此后，便是每个时钟周期流出一个结果，共有 n 个结果。

- 所有的操作时间都应时钟周期的整数倍，而且也不会在流水线中存在“瓶颈”，假设基本时间为 $T_c = \Delta t$ 。
- 式中的 s 和 e 表示前两段时间是时钟周期的多少倍，即需要多少个时间周期

4.4.1 向量指令的处理时间 T_{vp}

第 77 页

2. 一组向量指令的处理时间

- 对于一组向量指令而言，其执行时间主要取决于三个因素：

- 向量的长度
- 向量操作之间是否存在流水功能部件的使用冲突
- **数据的相关性（流水执行中断）**

- 把能在同一个时钟周期内一起开始执行的几条向量指令称为一个编队。

- 可以看出：同一个编队中的向量指令之间一定不存在流水向量功能部件的冲突和数据的冲突

- 编队后，这个向量指令序列的总的执行时间为各编队的执行时间的和

- $T_{vp}^{(i)}$ ：第 i 个编队的执行时间

- m ：编队的个数

$$T_{all} = \sum_{i=1}^m T_{vp}^{(i)}$$

4.4.1 向量指令的处理时间 T_{VP}

第 78 页

2. 一组向量指令的处理时间

- 影响一组向量指令运行时间的要素：（1）向量中元素的个数；（2）组织编队的情况
- 将能够组合在一起并行运行的指令称为一个“编队”
- 编组原则：

$$\begin{aligned} V_0 &\leftarrow V_1 + V_2 \\ V_3 &\leftarrow V_4 * V_5 \end{aligned}$$

(a) 不相关的指令

$$\begin{aligned} V_3 &\leftarrow V_1 + V_2 \\ V_6 &\leftarrow V_4 + V_5 \end{aligned}$$

(b) 加法功能部件的冲突

$$\begin{aligned} \mathbf{V_1} &\leftarrow V_3 + V_2 \\ V_6 &\leftarrow \mathbf{V_1} * V_5 \end{aligned}$$

(c) 操作数寄存器 V_1 的冲突

$$\begin{aligned} \mathbf{V_1} &\leftarrow V_3 + V_2 \\ V_6 &\leftarrow \mathbf{V_1} + V_5 \end{aligned}$$

(d) 加法功能部件和操作数寄存器 V_1 的冲突

4.4.1 向量指令的处理时间 T_{vp}

例4.4：下面是一组向量指令，假定每种指令都有一个功能流水线，而且不考虑向量链接，那么下面的一组向量指令能分成几个编队？

LOADV	V1, RX	//从存储器取向量RX到V1
MULTSV	V2, S0, V1	//标量S0乘向量V1，存入V2
LOADV	V3, RY	//从存储器取向量RY到V3
ADDV	V4, V2, V3	//向量V2加上向量V3，存入V4
STOREV	RY, V4	//向量V4保存RY中

- 分析：
- (1) 第二条指令的执行依赖第一条指令的执行结果，存在数据相关，不能成为一个编队
 - (2) 第一条指令与第三条指令之间发生了功能部件冲突，也不允许乱序执行
 - (3) 后面的两条指令必须利用前几条指令的运行结果，都不可能与其他指令组成编队

4个编队：

LOADV	; 第一编队
MULTSVLOADV	; 第二编队 (没有数据相关和功能部件冲突的两条指令)
ADDV	; 第三编队
STOREV	; 第四编队

4.4.1 向量指令的处理时间 T_{vp}

第 80 页

- 当一个编队是由若干条指令组成时，其执行时间就应该由该编队中各指令的执行时间的最大值来确定。

$T_{start}^{(i)}$ ：第 i 编队中各指令的启动时间的最大值

$$T_{all} = \sum_{i=1}^m T_{vp}^{(i)} = \sum_{i=1}^m (T_{start}^{(i)} + n)T_c = (\sum_{i=1}^m T_{start}^{(i)} + mn)T_c = (T_{start} + mn)T_c$$

$$T_{start} = \sum_{i=1}^m T_{start}^{(i)}$$

该组指令总的启动时间（时钟周期个数）

- 表示成时钟周期个数

$$T_{all} = T_{start} + mn \quad (\text{拍})$$

4.4.1 向量指令的处理时间 T_{VP}

例4.5 如果向量处理机各不同功能部件启动时的开销分别为：

- 取数和存数部件为12个时钟周期；
- 乘法部件为7个时钟周期；
- 加法部件为6个时钟周期。

请计算上例中每个编队的开始时间、获量第一个结果元素和最后一个结果元素的时间。

解：为了使问题讨论简单，假定向量处理机对向量长度n没有限制，即一条向量指令可以处理的向量元素是无穷多个

编队数 $m=4$

编队	开始时间	获得第一个结果时间	获得最后一个结果时间
LOADV	0	12	$11+n$
MULTSV LOADV	$12+n$	$12+n+12$	$23+2n$
ADDV	$24+2n$	$24+2n+6$	$29+3n$
STOREV	$30+3n$	$30+3n+12$	$41+4n$

4.4.1 向量指令的处理时间 T_{vp}

第 82 页

3. 分段开采时一组向量指令的总执行时间

- 当向量长度 n 大于向量寄存器长度 MVL 时，需要分段开采。
- 引入一些额外的处理操作

(假设：这些操作所引入的额外时间为 T_{loop} 个时钟周期)

$$\text{设 } \left\lfloor \frac{n}{MVL} \right\rfloor = p \quad q: \text{ 余数}$$

共有 m 个编队

- 对于最后一次循环来说，所需要的时间为：

$$T_{last} = T_{start} + T_{loop} + m \times q$$

4.4.1 向量指令的处理时间 T_{vp}

第 83 页

- 其他的每一次循环所要花费的时间为：

$$T_{\text{step}} = T_{\text{start}} + T_{\text{loop}} + m \times \text{MVL}$$

- 总的执行时间为：

$$\begin{aligned} T_{\text{all}} &= T_{\text{step}} \times p + T_{\text{last}} \\ &= (T_{\text{start}} + T_{\text{loop}} + m \times \text{MVL}) \times p + (T_{\text{start}} + T_{\text{loop}} + m \times q) \\ &= (p + 1) \times (T_{\text{start}} + T_{\text{loop}}) + m (\text{MVL} \times p + q) \\ &= \left\lceil \frac{n}{\text{MVL}} \right\rceil (T_{\text{start}} + T_{\text{loop}}) + mn \end{aligned}$$

4.4.1 向量指令的处理时间 T_{vp}

例4.6 有一台向量处理机，其寄存器的长度（MVL）为64个字，如果各功能流水线的启动时间与上题相同。假定 T_{loop} 为一个固定值15，向量长度为200。请计算为实现算式： $A=B*s$ 运算需要花费的时钟周期数。

解：为完成这个运算，需要编译成如下三个指令

LOADV	V1, RB	;取向量B
MULTSV	V2, V1, Fs	;向量B乘标量s，存入V2
STOREV	RA, V2	;存成向量A

向量长度 $n=200$
编队数 $m=3$
寄存器长度 $MVL=64$
启动时间 $T_{start}=12+7+12=31$ 个时钟周期
循环时间 $T_{loop}=15$

$$\begin{aligned} T &= \left\lceil \frac{n}{MVL} \right\rceil (T_{start} + T_{loop}) + mn \\ &= 4 \times (31+15) + 3 \times 200 \\ &= 784 \text{ 个时钟周期} \end{aligned}$$

4.4.1 向量指令的处理时间 T_{VP}

第 85 页

例4.7 在某向量处理机上执行DAXPY的向量指令序列，也即完成：

$$Y = \alpha \times X + Y$$

其中X和Y是向量，最初保存在主存中， α 是一个标量，已存放在寄存器F0中。它们的向量指令序列如下：

LV	V1, Rx
MULTFV	V2, F0, V1
LV	V3, Ry
ADDV	V4, V2, V3
SV	V4, Ry

4.4.1 向量指令的处理时间 T_{vp}

第 86 页

假设向量寄存器的长度 $MVL=64$ ， $T_{loop}=15$ ，各功能部件的启动时间为：

- 取数和存数部件为12个时钟周期；
- 乘法部件为7个时钟周期；
- 加法部件为6个时钟周期。

分别对于不采用向量链接技术和采用链接技术的两种情况，求完成上述向量操作的总执行时间。

4.4.1 向量指令的处理时间 T_{vp}

第 87 页

解：当不采用向量链接技术时，可以把上述五条向量指令分成4个编队：

- 第一编队：LV V1, Rx;
- 第二编队：MULTFV V2, F0, V1; LV V3, Ry;
- 第三编队：ADDV V4, V2, V3;
- 第四编队：SV V4, Ry。

$$T_{start} = 12 + 12 + 6 + 12, m = 4$$

可知，对 n 个向量元素进行DAXPY表达式计算所需的时钟周期个数为：

$$\begin{aligned} T_n &= \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + mn \\ &= \left\lceil \frac{n}{64} \right\rceil \times (15 + 12 + 12 + 6 + 12) + 4n = \left\lceil \frac{n}{64} \right\rceil \times 57 + 4n \end{aligned}$$

4.4.1 向量指令的处理时间 T_{VP}

第 88 页

采用向量链接技术，那么上述5条向量指令的编队结果如下 ($m=3$)

- 第一编队：LV V1,Rx; MULTFV V2,F0,V1;
- 第二编队：LV V3,Ry; ADDV V4,V2,V3;
- 第三编队：SV V4, Ry。

前两个编队中各自的两条向量指令都可以链接执行。根据链接的含义可知：

- 第一编队启动需要 $12+7=19$ 个时钟周期
- 第二个编队启动需要 $12+6=18$ 个时钟周期
- 第三个编队启动仍然需要12个时钟周期

对 n 个向量元素进行计算所需的时钟周期数为：

$$\begin{aligned} T_n &= \left\lceil \frac{n}{MVL} \right\rceil \times (T_{loop} + T_{start}) + mn \\ &= \left\lceil \frac{n}{64} \right\rceil \times (15 + 19 + 18 + 12) + 3n = \left\lceil \frac{n}{64} \right\rceil \times 64 + 3n \end{aligned}$$

4.4.2 最大性能 R_{∞} 和半性能向量长度 $n_{1/2}$ 第 89 页

1. 向量处理机的峰值性能 R_{∞}

- R_{∞} 表示当向量长度为无穷大时，向量处理机的最高性能，也称为峰值性能。

$$R_{\infty} = \lim_{n \rightarrow \infty} \frac{\text{向量指令序列中浮点运算次数} \times \text{时钟频率}}{\text{向量指令序列执行所需的时钟周期数}}$$

- 对于上述例题4.7 向量指令序列中的操作而言，只有“MULTFV V2,F0,V1”和“ADDV V4,V2,V3”两条浮点操作向量指令。
- 假设该向量处理机的时钟频率为200MHz，那么：

$$R_{\infty} = \lim_{n \rightarrow \infty} \frac{\text{向量指令序列中浮点运算次数} \times \text{时钟频率}}{\text{向量指令序列执行所需的时钟周期数}} = \lim_{n \rightarrow \infty} \frac{2 \times n \times 200}{\left\lceil \frac{n}{64} \right\rceil \times 64 + 3n} = \lim_{n \rightarrow \infty} \frac{2 \times n \times 200}{4n} = 100 \text{ MFLOPS}$$

4.4.2 最大性能 R_{∞} 和半性能向量长度 $n_{1/2}$ 第 90 页

2. 半性能向量长度 $n_{1/2}$

- 半性能向量长度 $n_{1/2}$ 是指向量处理机的性能为其最大性能的一半时所需的向量长度。
- 评价向量流水线的建立时间对性能影响的重要参数。

例4.8 对于例4.7，假设时钟频率为200MHz，求半性能向量长度 $n_{1/2}$ 。

假设该向量处理机的峰值性能 $R_{\infty} = 100$ MFLOPS，所以根据半性能向量长度的定义有：

假设 $n_{1/2} \leq 64$ ，那么有：

$$\frac{2 \times n_{1/2} \times 200}{\left\lceil \frac{n_{1/2}}{64} \right\rceil \times 64 + 3n_{1/2}} = 50$$
$$64 + 3n_{1/2} = \frac{2 \times n_{1/2} \times 200}{50} = 8n_{1/2}$$
$$5n_{1/2} = 64, \quad n_{1/2} = 12.8$$

$$n_{1/2} = 13$$

4.4.2 最大性能 R_∞ 和半性能向量长度 $n_{1/2}$ 第 91 页

3. 向量长度临界值 n_v

- 向量长度临界值 n_v 是指：对于某一计算任务而言，向量方式的处理速度优于标量串行方式处理速度时所需的最小向量长度。
- 对于上述DAXPY的例子
 - 假设，在标量串行工作方式下实现DAXPY循环的开销为10个时钟周期。那么在标量串行方式下，计算DAXPY循环所需要的时钟周期数为：

$$T_s = (10 + 12 + 12 + 7 + 6 + 12) \times n_v = 59n_v$$

- 在向量方式下，计算DAXPY循环所需要的时钟周期数为： $T_v = 64 + 3n_v$

- 根据向量长度临界值的定义，有 $T_v = T_s$
 $64 + 3n_v = 59n_v$ $n_v = \left\lceil \frac{64}{56} \right\rceil = 2$

4.5 向量处理机实例

4.5.1 具有代表性的向量处理机

第 93 页

■ 美国和日本生产的一些向量处理机的简要信息

系统型号	推出时间	最大配置, 时钟周期, 操作系统/编译系统	特色和要点
Cray 1S	1976年	有10条流水线的单处理机, 12.5ns, COS/CF7 2.1	第一台基于ECL的超级计算机
Cray 2S/4-256	1985年	256M字存储器的4台处理机, 4.1ns, COS或UNIX/CF77 3.0	16K字的本地存储器, 移植了UNIX V
Cray X-MP 416	1983年	16M字存储器的4台处理机, 128M字SSD, 8.5ns, COS/CF77 5.0	使用共享寄存器组用于IPC
Cray Y-MP 832	1988年	128M字存储器的8台处理机, 6ns, CF77 5.0	X-MP的改进型
Cray Y-MP C-90	1991年	每台处理机2条向量流水线, 16台处理机, 4.2ns, UNICOS/CF77 5.0	最大的Cray机器
CDC Cyber 205	1982年	有4条流水线的单处理机, 20ns, 虚拟OS/FTN200	存储器-存储器系统结构
ETA 10E	1985年	单处理机, 10.5ns, ETAV/FTN 200	Cyber 205的后继型号
NEC SX-X/44	1991年	每台处理机4组流水线, 4台处理机, 2.9ns, F77SX	
Fujitsu VP2600/10	1991年	5条流水线的单处理机和双标量处理机, 3.2ns, MSP.EX/F77 EX/VP	使用可重构微向量寄存器和屏蔽
Hitachi 820/80	1988年	512MB存储器, 18个流水线功能部件的单处理机, 4ns, FORT 77/HAP V23-OC	64个I/O通道, 最大传输率为288MB/秒

4.5.2 Cray Y-MP和C-90

第 94 页

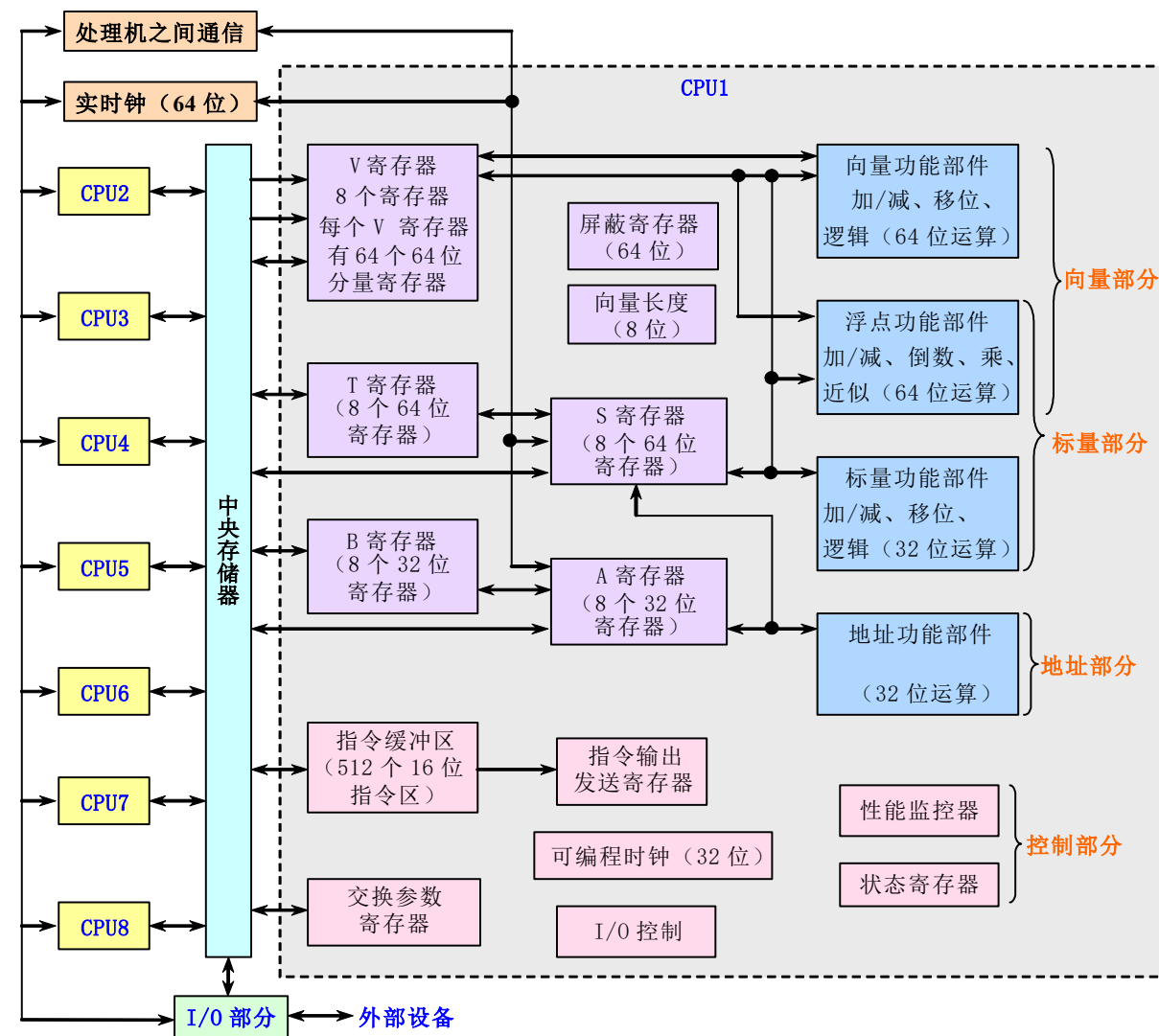
■ Cray Y-MP

➤ 系统结构

- ✓可以配置1台、2台、4台或8台处理机
- ✓8个CPU共享中央存储器、I/O 子系统、处理机通信子系统和实时时钟
- ✓CPU的时钟周期：6ns

➤ 中央存储器

- ✓分成256个交叉访问的存储体，通过每个CPU对4个存储器端口的交叉访问可以实现对存储器的重叠存取。
- ✓容量最大可达1GB。固态存储器的容量最大可达4GB。4个存储器访问端口允许每个CPU同时执行两个标量和向量取操作、一个存储操作和一个独立的I/O操作。



2. C-90

- 由16个类似于Y-MP的CPU组成
- 16台处理机共享主存储器的容量高达256M字（2GB）
- SSD存储器的容量最多达16GB
 - 可选作第二级主存储器
- 两条向量流水线和两个功能部件可以并行操作，每个时钟周期能产生4个向量计算结果。
 - 意味着每台处理机有4路并行性，因此16台处理机每个时钟周期最多可以产生64个向量计算结果。
- 运行UNICON操作系统，提供向量化的FORTRAN 77和C编译器。
- 64路并行性和4.2ns时钟周期相配合，可使系统的峰值性能达到16GFLOPS，系统最大I/O吞吐率为13.6MB/s。

4.5.3 NEC SX-X44

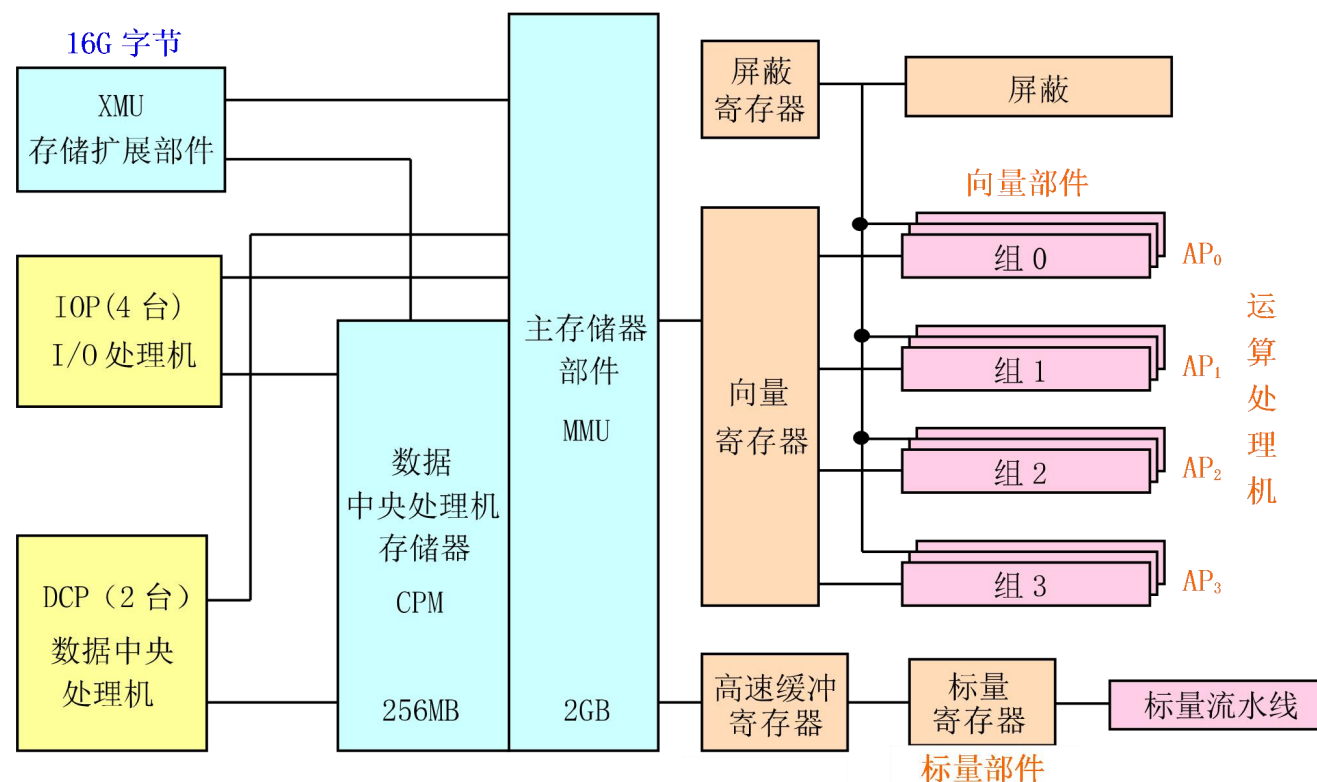
第 96 页

1. NEC 1991年推出 峰值速度可达
到22GFLOPS

- 主要措施之一：使用了基于VLSI和高密度封装技术的2.9ns的时钟

2. 4台运算处理机通过共享寄存器或通过2GB的共享存储器进行通信。

- 每台处理机有4组向量流水线
 - ✓ 每组包括2条加法/移位流水线和2条乘法/逻辑流水线
- 类似于C-90，4台处理机可达到64路并行



3. 高速标量部件

- 采用了具有128个标量寄存器的RISC系统结构，通过把指令重新排序来开发较高的并行性。

4. 主存储器为1024路的交叉访问存储器

- 其扩展存储器的最大容量高达16GB
- 最大传输率：2.75GB/s

5. 系统最多可以配置4台I/O处理机

- 每台I/O处理机的数据传输率：1GB/s
- 最多可以提供256个通道，用于高速网络、图形和外围操作，支持100MB/s的通道传输。

1. 向量的处理方式：横向处理方式、纵向处理方式、纵横向处理方式
2. 向量处理机的结构：存储器-存储器型结构、寄存器-寄存器型结构、CRAY-1的基本结构
3. 提高向量处理机性能的常用技术：设置多个功能部件、链接技术、分段开采技术、多处理机系统
4. 向量处理机的性能评价：向量指令的处理时间、向量处理机的最大性能、半性能向量长度、向量长度临界值
5. 向量处理机实例：Cray Y-MP和C-90、NEC SX-X44

选择题

第 99 页

1、Cray-1的流水线是 (**A**)

A. 多条单功能流水线

B. 一条单功能流水线

C. 多条多功能流水线

D. 一条多功能流水线

2、Cray-1向量处理机要实现指令间的链接，必须满足下列条件中的 (**C**)

A. 源向量相同，功能部件不冲突，有指令相关

B. 源向量不同，功能部件相同，无指令相关

C. 源向量、功能部件都不相同，指令有写后读冲突

D. 源向量、功能部件都不相同，指令有读后写冲突

选择题

第 100 页

3、Cray-1向量处理机启动存储器、流水部件及寄存器打入各需一拍，现有向量指令串：

$V_3 \leftarrow \text{存储器}$ (从存储器中取数：6拍)

$V_4 \leftarrow V_0 + V_1$ (向量加：6拍)

$V_5 \leftarrow V_3 \times V_4$ (向量乘：7拍)

向量长度均为N，则指令串最短的执行时间是 (**A**)

- A. 16+N 拍 B. 17+N 拍 C. 18+N 拍 D. 19+N 拍

4、Cray-1的两条向量指令属于 (**B**)

- A. 没有功能部件冲突和源向量冲突，可以并行
B. 没有功能部件冲突和源向量冲突，可以链接
C. 没有源向量冲突，可以交换执行顺序
D. 有向量冲突，只能串行

$$\begin{aligned} V_1 &\leftarrow V_2 + V_3 \\ V_4 &\leftarrow V_1 \times V_5 \end{aligned}$$

应用题

第 101 页

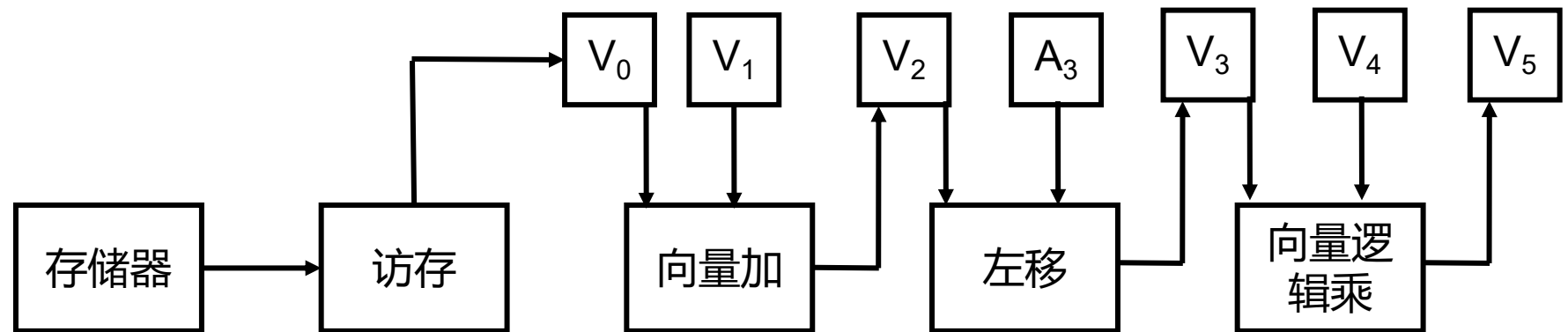
1、在Cray-1机器上，按照链接方式执行下述 4条向量指令（括号中给出了相应功能部件的执行时间），如果向量寄存器和功能部件之间的数据传送需要1拍，试求此链接流水线的通过时间是多少拍？如果向量长度为64，则需多少拍才能得到全部结果？

$V_0 \leftarrow \text{存储器}$

$V_2 \leftarrow V_0 + V_1$

$V_3 \leftarrow V_2 \ll A_3$

$V_5 \leftarrow V_3 \wedge V_4$



$$T_{\text{通过}} = (7+1) + (1+3+1) + (1+4+1) + (1+2+1) = 23 \text{ (拍)}$$

$$T_{\text{总共}} = T_{\text{通过}} + (64-1) = 23 + 63 = 86 \text{ (拍)}$$

2、在一台向量处理机上实现 $A=B \times S$ 计算，其中，A和B是长度为 $N=200$ 的向量，S是一个标量。向量寄存器长度 $MVL=64$ ，各功能部件的启动时间为：取数和存数部件为12个时钟周期、乘法部件为7个时间周期，执行标量代码的开销 $T_{loop}=15$ 个时钟周期，对一个向量元素执行一次操作的时间 $T_g=1$ 个时钟周期。求计算A的总执行时间。

解：假设向量A和B存放在向量寄存器Ra和Rb中，标量S存放在标量寄存器中，由下面3条指令完成计算

LV	V1, Rb
MULTSV	V2, R0, V1
SV	V2, Ra

这3条指令之间存在相关，需分为3个编队 $m=3$ ，同时向量需要分为 $[200/64]=4$ 组进行计算

$$T_{start}=12+7+12=31 \quad T_{loop}=15, \text{ 则}$$

$$T=4 \times (T_{loop}+T_{start}) + 3 \times 200 \times 1 = 784 \text{ 个时钟周期}$$

3、向量处理机Cray Y-MP/8的机器周期时间为6ns，一个周期可以完成一次加和一次乘法运算。另外，8台处理机在最好的情况下可以同时运行而互不干扰。计算Cray Y-MP/8的峰值性能。

解：峰值性能为

$$R_{\infty} = \frac{(1 + 1) \times 8}{T \times 10^6} = \frac{16}{6 \times 10^{-9} \times 10^6} \approx 2667 MFLOPS$$

4、在某向量处理机上执行DAXPY的向量指令序列，即完成 $Y=aX+Y$ 。其中，X和Y是向量，最初保存在主存中，a是一个标量，已存放在寄存器F0中。它们的向量指令如下：

LV	V1, Rx	//取向量X
MULTFV	V2, F0, V1	//向量X和标量（F0）相乘
LV	V3, Ry	//取向量Y
ADD	V4, V2, V3	//完成 $Y=aX+Y$
SV	V4, Ry	//存结果

假设向量寄存器的长度MVL=64，Tloop=15，各功能部件的启动时间为：

- (1) 取数和存数部件为12个时钟周期；
- (2) 乘法部件为7个时钟周期；
- (3) 加法部件为6个时钟周期。

分别对不采用向量链接技术和采用链接技术的两种情况，求完成上述向量操作的总执行时间。

解：一、不采用向量链接技术，上述5条向量指令分成4个编队：

(1) LV V1, Rx

(2) MULTFV V2, F0, V1 ; LV V3, Ry

(3) ADD V4, V2, V3

(4) SV V4, Ry

$$T_{\text{start}} = 12 + 12 + 6 + 12 = 42 \quad m=4$$

根据公式可知，对n个向量元素进行DAXPY表达式云计算所需的时钟周期个数为：

$$T_n = \left\lceil \frac{n}{MVL} \right\rceil \times (T_{\text{loop}} + T_{\text{start}}) + mn = \left\lceil \frac{n}{64} \right\rceil \times (15 + 42) + 4n = \left\lceil \frac{n}{64} \right\rceil \times 57 + 4n$$

解：二、采用向量链接技术，上述5条向量指令分成3个编队：

(1) LV V1, Rx; MULTFV V2, F0, V1

(2) LV V3, Ry; ADD V4, V2, V3

(3) SV V4, Ry

$$T_{\text{start}} = (12+7) + (12+6) + 12 = 49 \quad m=3$$

根据公式可知，对n个向量元素进行DAXPY表达式云计算所需的时钟周期个数为：

$$T_n = \left\lceil \frac{n}{MVL} \right\rceil \times (T_{\text{loop}} + T_{\text{start}}) + mn = \left\lceil \frac{n}{64} \right\rceil \times (15 + 49) + 3n = \left\lceil \frac{n}{64} \right\rceil \times 64 + 3n$$

第5章 指令级并级及其开发——硬件方法